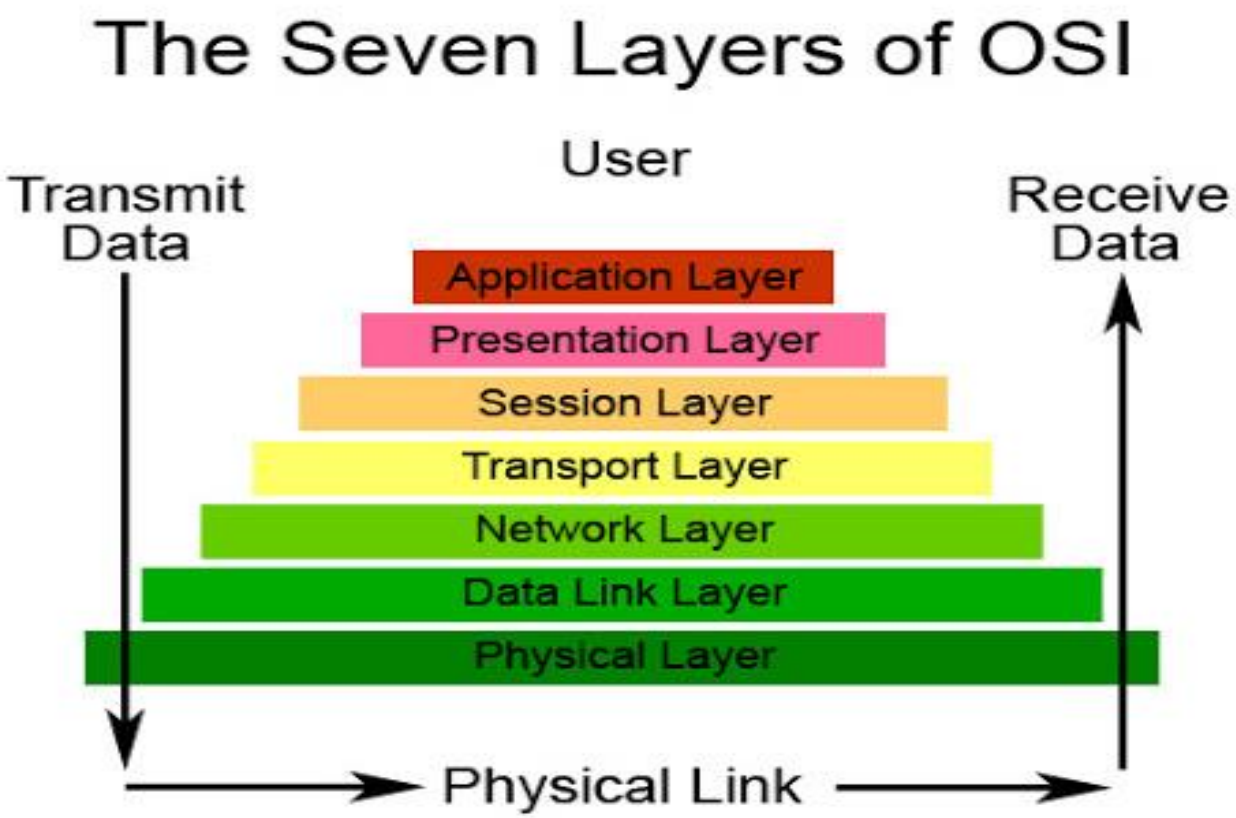
WEEK 1: Networking Fundamentals | Days: 1-7

Day 1 Tasks—OSI MODEL

➢ Draw the complete 7-layer OSI model diagram?

The OSI (Open Systems Interconnection) model is a conceptual framework developed by the International Organization for Standardization (ISO). It provides a structured approach to understanding how data travels from one device to another over a network.

Layer	Name Function	Example
7	Application	User interface HTTP, FTP, DNS
6	Presentation	Data translation, encryption SSL/TLS, JPEG
5	Session	Manage sessions NetBIOS, RPC
4	Transport	Reliable/unreliable delivery TCP, UDP
3	Network	Routing, logical addressing IP, and Router
2	Data Link	Physical addressing, error detection, Switch
1	Physical	Bits, signals, cables RJ45, Fiber



Layer 1: Physical Layer

This is the bottommost layer of the OSI reference model. It handles the physical connection between communicating devices.

- Responsible for transmitting raw bits from one node to the next.
- Upon receiving data, it converts incoming signals into binary form (0s and 1s) and forwards them to the Data Link layer.
- Typical devices operating at this layer include hubs, repeaters, modems, and various types of cabling.

Layer 2: Data Link Layer (DLL)

The Data Link Layer handles node-to-node message delivery. It ensures error-free data transfer between two directly connected nodes over the physical layer.

- When a packet reaches a network, the DLL is responsible for forwarding it to the correct host using the device's MAC address.
- At this layer, a packet is called a frame.
- Common DLL devices include switches and bridges.
- The packet received from the Network Layer is broken into smaller frames based on the frame size supported by the Network Interface Card (NIC).
- The DLL adds the sender and receiver MAC addresses into the frame header.

Sublayers of the Data Link Layer

- **Logical Link Control (LLC):** Coordinates communication between upper layers and handles error checking and flow control.-
- **Media Access Control (MAC):** Regulates how devices access the shared transmission medium and manages MAC addressing.

Layer 3: Network Layer

The Network Layer handles data transmission between hosts located in different networks. It is also responsible for routing packets across the network.

- The sender and receiver IP addresses are inserted into the header by the Network Layer.
- At this layer, a segment is referred to as a packet.
- The Network Layer is implemented using networking devices such as routers and layer-3 switches.

Handles logical addressing and routing of packets.

- Uses IP addresses for routing decisions
- Determines the best path to the destination
- Manages packet forwarding and routing tables

Layer 4: Transport Layer

The Transport Layer ensures end-to-end message delivery. It provides services to the Application Layer and receives services from the Network Layer.

- Data at this layer is called a segment.
- The Transport Layer guarantees successful data transmission.
- Common Transport Layer protocols include TCP, UDP,NetBIOS and PPTP
- It adds source and destination port numbers to the segment header and forwards the segmented data to the Network Layer.

Ensures reliable or fast delivery of data.

- Uses TCP (reliable) and UDP (fast, connectionless)
- Handles segmentation and reassembly of data
- Provides flow control (sliding window)
- Ensures error recovery and end-to-end communication

Layer 5: Session Layer

The Session Layer manages the establishment, maintenance, and termination of communication sessions between two devices. It also provides authentication and security features.

- Establishes, manages, and terminates sessions.
- Controls dialog between devices (duplex/half-duplex).
- Maintains session checkpoints and recovery.
- Handles session authentication and reconnection.
- Used in RPC, SQL sessions, NetBIOS.

Layer 6: Presentation Layer

The Presentation Layer is also known as the Translation Layer. It extracts data from the Application Layer and transforms it into the appropriate format for network transmission.

- Common Presentation Layer protocols include TLS/SSL (Transport Layer Security / Secure Sockets Layer).
- Formats such as JPEG, MPEG, and GIF are encoding standards managed by this layer.

Translates, encrypts, and formats data.

- Converts data formats (encoding/decoding).
- Applies encryption and decryption (SSL/TLS).
- Handles compression for efficient transmission.
- Makes data readable for the application layer.

Layer 7: Application Layer

The Application Layer sits at the top of the OSI model stack. It is implemented by network applications that generate the data to be sent across the network.

- This layer acts as a gateway for application services to access the network and display information.
- Supports protocols like HTTP, FTP, SMTP, and DNS.
- Provides network services such as email, web access, and file transfer.
- Interacts directly with user applications.
- Identifies communication partners and resource availability

Data Flow in the OSI Model

When information is transferred from one device to another, it passes through all seven layers of the OSI model. First, data travels downward through the seven layers at the sender's side, then climbs back up the seven layers at the receiver's side.

Step-by-Step Data Flow

Layer	Action
Application Layer	Applications generate the data.
Presentation Layer	Data is formatted and encrypted.
Session Layer	Connections are established and managed.
Transport Layer	Data is split into segments for reliable delivery.
Network Layer	Segments are packaged into packets and routed.
Data Link Layer	Packets are framed and sent to the next device.
Physical Layer	Frames are converted into bits and physically transmitted.

Note: Each layer adds specific information to help the data reach its destination correctly. These steps are reversed when the data arrives at the receiving device.

Practical Example:

Sending an Email

Let us walk through an example of Person A sending an email to Person B.

Step 1 (Application Layer): Person A interacts with an email application such as Gmail or Outlook and composes the message.

Step 2 (Presentation Layer): The email application prepares the data for transmission by encrypting it and formatting it appropriately.

Step 3 (Session Layer): A connection is established between the sender and receiver over the internet.

Step 4 (Transport Layer): The email data is broken into smaller segments. Sequence numbers and error-checking information are added to ensure reliability.

Step 5 (Network Layer): Packets are addressed and the best route for transfer is determined.

Step 6 (Data Link Layer): Data packets are encapsulated into frames. MAC addresses are added for local devices, and error detection is performed.

Step 7 (Physical Layer): Frames are transmitted as electrical or optical signals over a physical network medium, such as Ethernet cable or Wi-Fi.

Once the email reaches Person B (the receiver), the entire process is reversed. The data is decrypted, reassembled, and finally displayed in Person B's email client.

- **Map real-world examples to each layer (e.g. HTTP → Application, Ethernet → Data Link).**

OSI Model Real-World Mapping

The OSI model offers a seven-layer structure for understanding networking. It connects practical technologies from user-facing applications down to physical hardware into a coherent framework.

Below are real-world examples assigned to each OSI layer.

Layer 7: Application Layer (User Interface)

What it does: Provides the interface through which users interact with network services.

Real-world examples:

HTTP/HTTPS (web browsing), SMTP/IMAP (email), FTP (file transfers), SSH (remote login), DNS (domain name resolution), WhatsApp Web.

Scenario:

Opening a browser and typing `google.com` — this action uses HTTP/HTTPS.

Layer 6: Presentation Layer (Data Formatting)

What it does: Handles data translation, encryption, and compression so information can be understood by the receiving application.

Real-world examples:

SSL/TLS (encryption and security), JPEG/PNG (image formats), MP4/AAC (video and audio), JSON/XML (data structuring), ASCII (character encoding)

Scenario:

When you visit a banking website, your browser automatically decrypts the incoming data using TLS.

Layer 5: Session Layer (Session Management)

What it does: Establishes, maintains, and terminates communication sessions between two devices.

Real-world examples:

RDP (Remote Desktop Protocol), SQL queries, NetBIOS, web session IDs (used to keep users logged in).

Scenario:

Joining a Zoom meeting and keeping the audio/video stream active throughout the call.

Layer 4: Transport Layer (End-to-End Delivery)

What it does: Ensures complete data transfer with appropriate reliability and speed.

Real-world examples:

TCP (reliable delivery for web traffic and email), UDP (fast delivery for streaming, gaming, and VoIP).

Scenario:

WhatsApp voice calling uses UDP — speed is prioritized over perfect packet delivery.

Layer 3: Network Layer (Routing)

What it does: Determines the best path to send data from source to destination across different networks.

Real-world examples:

IP (IPv4 and IPv6), ICMP (used by ping), OSPF, BGP (Border Gateway Protocol).

Scenario:

Your home router uses IP addresses to route a Netflix data packet to your television.

Layer 2: Data Link Layer (Physical Addressing)

What it does: Transfers data between nodes on the same local network using hardware addresses.

Real-world examples:

Ethernet (802.3), Wi-Fi (802.11), MAC addresses, network switches, ARP (Address Resolution Protocol).

Scenario:

An Ethernet switch uses MAC addresses to forward data only to the intended device on your local network.

Layer 1: Physical Layer (Hardware Transmission)

What it does: Transmits raw bits (0s and 1s) over a physical medium.

Real-world examples:

Ethernet cables (Cat5e, Cat6), fiber optic cables, Wi-Fi radio waves, Bluetooth, USB connectors, hubs, network interface cards (NICs).

Scenario:

A Wi-Fi router sends radio signals through the air to your laptop.

Summary Table: Web Request Example (`https: //google.com)

OSI Layer	Component	Real-World Activity
7. Application	HTTP/HTTPS	Browser requests the webpage
6. Presentation	TLS/SSL	Data is encrypted before transmission
5. Session	Session ID	Connection state is maintained during the request
4. Transport	TCP	Reliable delivery handshake occurs (SYN/ACK)
3. Network	IP	Packets are routed using IP addresses
2. Data Link	Ethernet/Wi-Fi	Data is framed and MAC addresses are added
1. Physical	Cables/Radio	Bits are transmitted as electrical or light signals

➢ TASK: Functions of each OSI layer.

Here are the core functions of each OSI layer, presented in a clear, original, and professional manner.

Functions of Each OSI Layer (Layers 1–7)

Layer 1: Physical Layer

Primary role: Transmission of raw bits over a physical medium.

Key functions:

- Bit encoding and signaling (converting 0s and 1s into electrical, optical, or radio signals).
- Defining physical characteristics of cables, connectors, and interfaces.
- Managing data rates (transmission speed).
- Synchronizing sender and receiver clocks.
- Specifying transmission direction (simplex, half-duplex, full-duplex).

Layer 2: Data Link Layer

Primary role: Reliable node-to-node data transfer across a single network link.

Key functions:

- **Framing:** Grouping bits into frames with defined start and end boundaries.
- **Physical addressing:** Adding source and destination MAC addresses to frames.
- **Error detection and correction:** Identifying and retransmitting corrupted or lost frames.
- **Flow control:** Preventing a fast sender from overwhelming a slow receiver.
- **Media access control:** Determining which device can transmit on a shared medium.

Layer 3: Network Layer

- **Primary role**: End-to-end delivery of packets across multiple networks.

- **Key functions:**
- **Routing:** Selecting the best path from source to destination
- **Logical addressing:** Assigning unique IP addresses to identify devices across different networks
- **Packet forwarding:** Moving packets from one router to the next toward the destination
- **Fragmentation and reassembly:** Breaking packets into smaller units when needed
- **Error handling and diagnostics:** Using protocols like ICMP for network troubleshooting

Layer 4: Transport Layer

Primary role: Reliable, end-to-end communication between processes on different hosts.

Key functions:

- **Segmentation and reassembly:** Dividing application data into smaller segments and reassembling them at the destination.
- **Port addressing (service point addressing):** Using port numbers to deliver data to the correct application.
- **Connection management:** Establishing, maintaining, and terminating connections (TCP only).
- **Reliability control:** Acknowledging received data and retransmitting lost segments.
- **Flow control (end-to-end):** Preventing buffer overflow at the receiver side.

Layer 5: Session Layer

Primary role: Managing communication sessions between applications

Key functions:

- **Session establishment, maintenance, and termination:** Setting up and closing active communication sessions.
- **Synchronization:** Inserting checkpoints (sync points) into data streams to allow recovery after interruptions.
- **Dialog control:** Determining whose turn it is to transmit (half-duplex or full-duplex).
- **Session restoration:** Recovering interrupted sessions without restarting from scratch.
- **Activity management:** Grouping related exchanges into logical activities.

Layer 6: Presentation Layer

Primary role: Translating data between application format and network format.

Key functions:

- **Data translation:** Converting between different character encodings (e.g., ASCII, EBCDIC).
- **Encryption and decryption:** Securing data for transmission and decrypting upon arrival.
- **Compression and decompression:** Reducing the number of bits transmitted to save bandwidth.
- **Data formatting:** Structuring data using standards like JSON, XML, or binary formats.
- **Protocol conversion:** Translating between different application-layer protocols when needed.

Layer 7: Application Layer

Primary role: Providing network services directly to end-user applications.

Key functions:

- **User interface:** Allowing applications to request network services.
- **Resource sharing and access:** Enabling remote file, printer, and database access.
- **Email services:** Supporting message composition, transfer, and storage.
- **Directory services:** Providing name-to-address resolution (e.g., DNS).
- **Remote login:** Allowing users to access remote systems securely (e.g., SSH, Telnet).
- **File transfer:** Moving files between systems (e.g., FTP, SFTP).
- **Web services:** Delivering web pages and APIs (e.g., HTTP/HTTPS)

➢ TASK: Data flow process (encapsulation & decapsulation).

Data Encapsulation

Data Encapsulation is the process in which some extra information is added to the data item to add some features to it. We use either the OSI or the TCP/IP model in our network, and the data transmission takes place through various layers in these models. Data encapsulation adds the protocol information to the data so that data transmission can take place in a proper way. This information can either be added in the header or the footer of the data. The data is encapsulated on the sender's side, starting from the application layer to the physical layer. Each layer takes the encapsulated data from the previous layer and adds some more information to encapsulate it and some more functionalities with the data. These functionalities may include proper data sequencing, error detection and control, flow control, congestion control, routing information, etc.

Data De-encapsulation

Data De-encapsulation is the reverse process of data encapsulation. The encapsulated information is removed from the received data to obtain the original data. This process takes place at the receiver's end. The data is de-encapsulated at the same layer at the receiver's end to the encapsulated layer at the sender's end. The added header and trailer information are removed from the data in this process. The below diagram shows how header and footer are added and removed from the data in the process of encapsulation and de-encapsulation respectively. The data is encapsulated in every layer at the sender's side and also de-encapsulated in the same layer at the receiver's end of the OSI or TCP/IP model. Actually, we use different terms for the encapsulated form of the data that is described in the below mentioned diagram. Now, we will learn the whole process of encapsulation and de-encapsulation in the OSI and TCP/IP model step-by-step as mentioned in the below picture.

Encapsulation Process (At sender's side)

1. Step 1: The Application, Presentation, and Session layer in the OSI model, or the Application layer in the TCP/IP model takes the user data in the form of data streams, encapsulates it and forwards the data to the Transport layer. It does not necessarily add any header or footer to the data. But it is application-specific and can add the header if needed

2. Step 2: The Transport layer (in the OSI or TCP/IP model) takes the data stream from the upper layers, and divide it into multiple pieces. The Transport layer encapsulates the data by adding the appropriate header to each piece. These data pieces are now called as data segments. The header contains the sequencing information so that the data segments can be reassembled at the receiver's end.

3. Step 3: The Network layer (in the OSI model) or the Internet layer (in the TCP/IP model) takes the data segments from the Transport layer and encapsulate it by adding an additional header to the data segment. This data header contains all the routing information for the proper delivery of the data. Here, the encapsulated data is termed as a data packet or datagram.

4. Step 4: The Data-Link layer (in the OSI or TCP/IP model) takes the data packet or datagram from the Network layer and encapsulate it by adding an additional header and footer to the data packet or datagram. The header contains all the switching information for the proper delivery of the data to the appropriate hardware components, and the trailer contains all the information related to error detection and control. Here, the encapsulated data is termed as a data frame.

5. Step 5: The Physical layer (in the OSI or TCP/IP model) takes the data frames from the Data-Link layer and encapsulate it by converting it to appropriate data signals or bits (corresponding to the physical medium).

De-Encapsulation Process (At receiver's side)

1. Step 1: The Physical layer (in the OSI or TCP/IP model) takes the encapsulated data signals or bits from the sender, and de-encapsulate it in the form of a data frame to be forwarded to the upper layer, i.e., the Data-Link layer.

2. Step 2: The Data-Link layer (in the OSI or TCP/IP model) takes the data frames from the Physical layer. It de-encapsulates the data frames and checks the frame header whether the data frame is switched to the correct hardware or not. If the frame is switched to the incorrect destination, it is discarded, else it checks the trailer information. If there is any error in the data, data retransmission is requested, else it is de-encapsulated and the data packet is forwarded to the upper layer. \

3. Step 3: The Network layer (in the OSI model) or the Internet layer (in the TCP/IP model) takes the data packet or datagram from the Data-Link layer. It de encapsulates the data packets and checks the packet header whether the packet is routed to the correct destination or not. If the packet is routed to the incorrect destination, the packet is discarded, else it is de-encapsulated and the data segment is forwarded to the upper layer.

4. Step 4: The Transport layer (in the OSI or TCP/IP model) takes the data segments from the network layer and de-encapsulate it. It first checks the segment header and then reassembles the data segments to form data streams, and these data streams are then forwarded to the upper layers.

5. Step 5: The Application, Presentation, and Session layer in the OSI model, or the Application layer in the TCP/IP model takes encapsulated data from the Transport layer, de-encapsulate it, and the application-specific data is forwarded to the applications. This is all about the data encapsulation and de-encapsulation in computer.

Data Movement in the OSI Model: Adding and Removing Headers

When information travels between two devices over a network, it must pass through all seven OSI layers. This journey consists of two opposite but related processes:

- **Encapsulation (transmitter side):** Each layer wraps the incoming data with its own control information (mostly headers, sometimes trailers) before passing it downward.
- **Decapsulation (receiver side):** Each layer strips off its corresponding control information as data moves upward toward the user application.

Encapsulation – Building the Data Package for Transmission

Encapsulation starts when an application generates data. As that data descends through the layers, each layer adds a specific piece of control information.

Detailed Encapsulation Steps

Phase	Layer	What Happens	Resulting Data Unit
1	Application (7)	the user's application creates the original payload	Data
2	Presentation (6)	the data may be encrypted, compressed, or reformatted	Data
3	Session (5)	Session identifiers and synchronization markers are inserted	Data
4	Transport (4)	the data stream is split into smaller pieces attached to each piece	Segment
5	Network (3)	a network header is prefixed to each segment	Packet.
6	Data Link (2)	A data link header is added to the front, and a trailer is attached to the end	Frame
7	Physical (1)	the complete frame is turned into a stream of bits and sent across the medium	Bits

Visual of Encapsulation

Sender – Top to Bottom

Original application data

[Transport Header | Original Data] ← This is a segment

[Network Header | Transport Header | Original Data] ← This is a packet

[Data Link Header | Network Header | Transport Header | Original Data | Data Link Trailer] ← this is a frame

[1 0 1 1 0 0 1 0 1 1 0 0 ...] ← Bits on the wire

Part 2: Decapsulation – Unwrapping the Data at the Destination

Decapsulation begins when the receiving device picks up the bits from the physical medium. As the data climbs upward, each layer removes the header (and trailer) that its counterpart added on the sender side.

Detailed Decapsulation Steps

Physical (1):

The device reads incoming bits and groups them into a frame; the frame is then handed to the Data Link Layer.

Data Link (2): The layer examines and removes the data link header and trailer; it verifies the CRC for errors; the remaining packet is passed upward to the Network Layer.

Network (3): The network header is read and stripped away; the destination IP address is checked; the extracted segment is sent to the Transport Layer

Transport (4): The transport header is processed and removed; segments are reassembled into the correct order; missing or damaged data is identified; the recovered data is passed to the Session Layer.

Session (5): Session control fields are read and removed; synchronization points are handled; the data continues upward to the Presentation Layer.

Presentation (6): The data is decrypted, decompressed, or converted back to its original format as needed; clean data is delivered to the Application Layer.

Application (7): The original message is finally presented to the user application.

Visual of Decapsulation

Receiver – Bottom to Top

Incoming bits from the network medium

[Data Link Header | Network Header | Transport Header | Original Data | Data Link Trailer] ← Frame

(Data Link Layer removes header + trailer)

[Network Header | Transport Header | Original Data] ← Packet

(Network Layer removes header)

[Transport Header | Original Data] ← Segment

(Transport Layer removes header)

Original application data

User application receives the data

Definition and Core Concepts

Encapsulation is the process where a sender adds headers and trailers to data units as they move down the Open Systems Interconnection (OSI) or Transmission Control Protocol/Internet Protocol (TCP/IP) model layers. Each layer creates a new, larger Protocol Data Unit (PDU) by wrapping the previous layer's data with its own control information. Decapsulation performs the inverse operation. The receiver removes headers and trailers from data units as they move up the network stack layers. This reveals the original data payload for processing by the next higher layer.

OSI and TCP/IP Models

The OSI model provides a seven-layer conceptual framework for network communication. The TCP/IP model uses four layers that map to specific OSI layers. Both models define how encapsulation and decapsulation occur at each protocol level.

Protocol Headers and Trailers

Headers contain control information specific to each layer's protocol. This includes addressing information, sequence numbers, flags, and error detection codes. Trailers appear primarily at the Data Link Layer for error checking through mechanisms like Frame Check Sequence (FCS).

Protocol Data Units

Each network layer uses specific terminology for its data unit:

- Application Layer: Data or Message
- Transport Layer: Segment (TCP) or Datagram (UDP)
- Network Layer: Packet
- Data Link Layer: Frame
- Physical Layer: Bits

How Encapsulation Works

Encapsulation begins at the Application Layer when a user application generates data for transmission. The data moves down through each layer of the network stack, with each layer adding its specific header information.

Application Layer Processing

The application creates raw data such as HTTP requests, email messages, or file transfers. This data contains no network addressing or transport information.

Transport Layer Addition

The Transport Layer adds either a TCP or UDP header. TCP headers include source and destination port numbers, sequence numbers, acknowledgment numbers, window size, and control flags. UDP headers contain only source port, destination port, length, and checksum fields.

Network Layer Wrapping

The Network Layer encapsulates the transport segment within an IP header. This header contains source and destination IP addresses, Time to Live (TTL) values, protocol identifiers, and fragmentation information. The resulting PDU becomes an IP packet.

Data Link Layer Framing

The Data Link Layer creates a frame by adding both a header and trailer. The header includes source and destination Media Access Control (MAC) addresses and frame type information. The trailer contains the Frame Check Sequence for error detection.

Physical Layer Transmission

The Physical Layer converts the complete frame into electrical signals, light pulses, or radio waves for transmission across the physical medium.

How Decapsulation Works

Decapsulation reverses the encapsulation process when data arrives at the destination device. The receiving network stack processes the data from the Physical Layer upward through each protocol layer.

Physical Layer Reception

The Physical Layer receives the transmitted signals and converts them back into digital frame data for processing by higher layers.

Data Link Layer Processing

The Data Link Layer examines the frame header to verify the destination MAC address matches the receiving interface. It performs error checking using the FCS trailer and removes both the header and trailer before passing the packet to the Network Layer.

Network Layer Examination

The Network Layer processes the IP header to verify the destination IP address. It checks TTL values, handles fragmentation if necessary, and removes the IP header before passing the segment to the Transport Layer.

Transport Layer Handling

The Transport Layer examines port numbers to determine the destination application. For TCP connections, it handles sequence numbers, acknowledgments, and flow control. The layer removes the transport header and delivers the data to the appropriate application.

Application Layer Delivery

The Application Layer receives the original data in its unmodified form for processing by the end-user application.

Key Features and Components

Several critical components enable encapsulation and decapsulation processes to function reliably across diverse network environments.

Sender and Receiver Endpoints

Network devices act as senders when transmitting data and receivers when processing incoming transmissions. Each device must implement complete network stacks to handle both encapsulation and decapsulation operations.

Network Stack Implementation

Operating systems implement network stacks as software modules that correspond to OSI or TCP/IP model layers. These modules interact through defined interfaces to process data units appropriately.

Header Information Fields

Headers contain essential control information:

- **Addressing**: Source and destination identifiers at each layer.
- **Sequencing**: Order information for reliable delivery.
- **Control Flags**: Protocol-specific operational parameters.
- **Error Detection**: Checksums and validation codes.

Trailer Components

Trailers primarily appear at the Data Link Layer and contain Frame Check Sequence values calculated from the entire frame contents. Some protocols use trailers for additional error detection or security information.

Use Cases and Applications

Encapsulation and decapsulation enable all forms of network communication through their fundamental role in data transmission protocols.

Standard TCP/IP Communication

Web browsing, email, file transfers, and remote access all rely on these processes. Each application generates data that undergoes complete encapsulation for transmission and decapsulation upon receipt.

Day 2 Tasks— TCP/IP Basics

- **Task: ping google.com and observe the output.**

Window Cmd Output

```
Microsoft Windows [Version 10.0.19045.6937]
(c) Microsoft Corporation. All rights reserved.

C:\Users\SMI>ping www.google.com

Pinging www.google.com [142.251.151.119] with 32 bytes of data:
Reply from 142.251.151.119: bytes=32 time=880ms TTL=114
Reply from 142.251.151.119: bytes=32 time=296ms TTL=114
Reply from 142.251.151.119: bytes=32 time=595ms TTL=114
Reply from 142.251.151.119: bytes=32 time=467ms TTL=114

Ping statistics for 142.251.151.119:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 296ms, Maximum = 880ms, Average = 559ms

C:\Users\SMI>_
```

Kali Linux Terminal Output

```
┌──(root㉿kali)-[/home/kali/Desktop]
└─# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=907 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=1198 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=5455 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=128 time=4472 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=128 time=3464 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=128 time=2441 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=128 time=1417 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=128 time=393 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=128 time=2518 ms
```

➢ TASK: Tracert (Windows) / traceroute (Linux) to any domain?

Window Cmd Output

```
C:\Users\SMI>tracert 8.8.8.8

Tracing route to dns.google [8.8.8.8]
over a maximum of 30 hops:

  1      3 ms      1 ms      1 ms  10.71.253.25
  2      *         *         *     Request timed out.
  3    190 ms    132 ms    262 ms  10.75.10.1
  4     68 ms     67 ms     75 ms  10.180.26.62
  5     50 ms     92 ms     62 ms  10.81.91.34
  6     96 ms     54 ms     65 ms  10.81.191.21
  7    119 ms     62 ms    105 ms  110.93.202.104
  8    111 ms     98 ms    124 ms  110.93.254.110
  9    197 ms     50 ms     66 ms  110.93.252.224
 10     94 ms     91 ms    353 ms  110.93.192.35
 11    113 ms    104 ms    117 ms  209.85.241.201
 12    117 ms    112 ms    104 ms  142.251.50.213
 13     94 ms     76 ms     90 ms  dns.google [8.8.8.8]

Trace complete.

C:\Users\SMI>
C:\Users\SMI>tracert 10.71.253.8

Tracing route to ASAD [10.71.253.8]
over a maximum of 30 hops:

  1     <1 ms     <1 ms     <1 ms  ASAD [10.71.253.8]

Trace complete.
```

Kali Linux terminal Output

```
┌──(root㉿kali)-[/home/kali/Desktop]
└─# traceroute google.com
traceroute to google.com (142.251.37.142), 30 hops max, 60 byte packets
 1  192.168.117.2 (192.168.117.2)  0.583 ms  0.256 ms  0.236 ms
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  * * *
12  * *
```

➢ TASK: Differences between TCP and UDP?

Answer:

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are two core protocols of the Transport Layer of the OSI and TCP/IP models. Both are responsible for end-to-end communication between applications, but they differ significantly in terms of reliability, speed, and use cases. Understanding the difference between TCP and UDP is essential for designing efficient and reliable networked systems. Protocols are rules that govern how data is formatted and sent over a network. TCP and UDP are two different methods for doing the same job: transferring data via the internet. They enable servers and devices to communicate so you can send emails, watch Netflix, play games, and browse web pages.

Transmission Control Protocol (TCP)

TCP is a reliable, connection-oriented transport protocol that ensures accurate and ordered data delivery. It uses control mechanisms to guarantee data correctness, which makes it slower but dependable.

- Connection-oriented protocol
- Reliable and ordered data delivery
- Higher overhead but high accuracy

User Datagram Protocol (UDP)

UDP is a fast, connectionless transport protocol that sends data without reliability guarantees. It is efficient for applications where speed is more important than accuracy.

- Connectionless and lightweight
- No guarantee of delivery or order
- Low overhead and high speed

Differences between TCP and UDP

TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection-oriented; uses a three-way handshake	Connectionless; no handshake
Guarantees reliable data delivery	Does not guarantee delivery
Uses acknowledgements (ACKs)	No acknowledgements
Supports retransmission of lost packets	No retransmission support
Ensures packets are delivered in order	Does not ensure ordering
Provides flow control and congestion control	No flow or congestion control
Slower due to higher overhead	Faster with minimal overhead
Variable header size (20–60 bytes)	Fixed header size (8 bytes)
Treats data as a continuous byte stream	Treats data as independent messages
Does not support broadcasting or multicasting	Supports broadcasting and multicasting
Used by HTTP, HTTPS, FTP, SMTP	Used by DNS, DHCP, VoIP, Streaming

Which protocol is better: TCP or UDP?

It depends on what you're doing online and the type of data being transferred. UDP is better if you're gaming online, because its speedy data transfer allows for mostly lag free gaming. TCP is better if you're transferring files, like family photos, because it ensures the data arrives exactly as it was sent. Overall, TCP and UDP are both useful protocols, so to think in terms of TCP vs UDP is a bit misleading. But depending on the type of data transfer, TCP or UDP might be better for the job. Here are some examples:

TCP is best for:

Email or texting
File transfers
Web browsing

UDP is best for:

live streaming
online gaming
Video chat

➢ TASK:Layer TCP/IP model vs 7-layer OSI model.

The 7-layer OSI model is a theoretical, vendor-neutral framework for understanding network communication, while the 4-layer TCP/IP model is the practical, protocol-specific implementation that powers the internet. OSI offers detailed separation of layers (Application-Physical), whereas TCP/IP combines similar functions (e.g., Application/Presentation/Session combined) for efficiency.

Key Comparisons:

- **Structure:** OSI has 7 layers (Physical, Data Link, Network, Transport, Session, Presentation, and Application); TCP/IP has 4 layers (Network Access, Internet, Transport, and Application).
- **Application & Use:** OSI is mainly used for teaching/reference, while TCP/IP is the actual protocol suite used for modern networking.
- **Layer Consolidation:**
  - OSI Layers 5-7 (Session, Presentation, and Application) are merged into the TCP/IP Application Layer.
  - OSI Layers 1-2 (Physical, Data Link) are merged into the TCP/IP Network Access Layer.
- **Reliability:** TCP/IP model supports only connectionless communication at the Network Layer, but both connection-oriented and connectionless at the Transport Layer. The OSI model supports both at the Network Layer.

Comparison Table:

Layer #	OSI Model	TCP/IP Model	Main Responsibility
7	Application	**Application**	Network apps (HTTP, FTP, DNS)
6	Presentation	**Application**	Encryption, formatting
5	Session	**Application**	Managing sessions
4	Transport	**Transport**	End-to-end connections (TCP/UDP)
3	Network	**Internet**	Routing, logical addressing (IP)
2	Data Link	**Network Access**	MAC addressing, framing
1	Physical	**Network Access**	Raw transmission (cables, bits)

Why the Differences Matter:

- **Troubleshooting:** The 7-layer OSI model is better for isolating problems because its layers are more specific.
- **Implementation:** TCP/IP is the protocol suite designed before the model itself, making it the practical choice for building networks.

Key Differences

Aspect	OSI Model	TCP/IP Model
Layers	7 layers	4 layers
Development	International standard (ISO)	Practical implementation (DoD)
Approach	Theoretical, "what to do"	Practical, "how it works"
Protocols	Protocol-independent	built around TCP/IP suite
Usage	Teaching, documentation	Actual internet implementation

Which One Matters?

TCP/IP Used on the actual internet. What engineers work with daily?

OSI Useful for conceptual understanding, troubleshooting, and certification exams.

OSI Layers (Bottom to Top).

Physical, Data Link, Network, Transport, Session, Presentation, Application.

TCP/IP Layers:

- All Internet Traffic Needs (Application, Internet, Transport, Network Access).

When to Use Each

Scenario	Use OSI	Use TCP/IP
• Real-world networking	✗	✓
• Studying for exams	✓	✓
• Troubleshooting conceptually	✓	✗
• Configuring routers	✗	✓
• Learning network fundamentals	✓	✗

The OSI model is great for understanding concepts, but TCP/IP is what actually powers the internet.

Day 3 Tasks—IP Addressing

- **TASK: Check your machine's IP using ipconfig (Windows) / Ifconfig (Linux)?**

IP Config Output

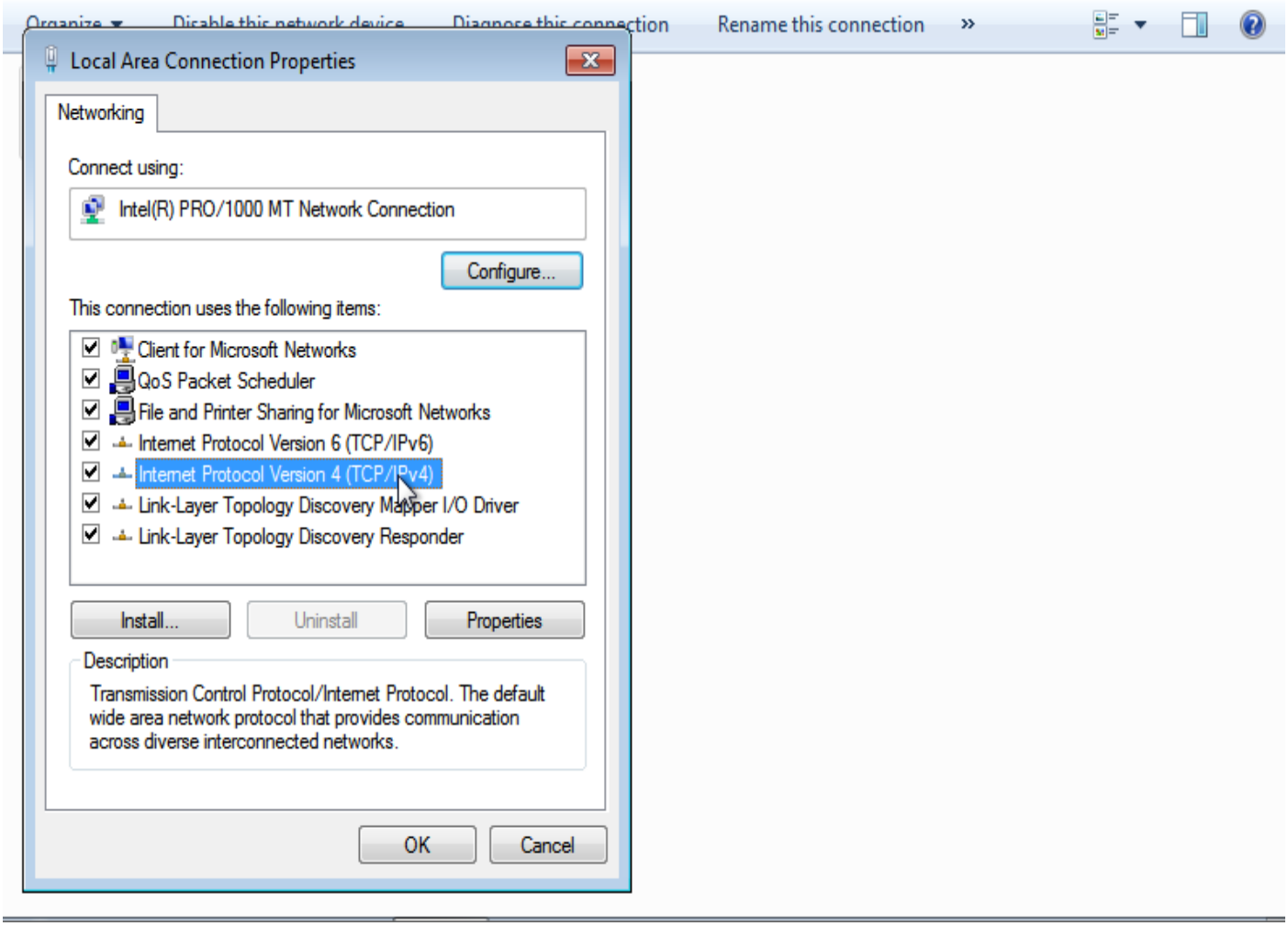
```
C:\Users\SMI>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Ethernet adapter vEthernet (Default Switch):

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::7818:31c5:5cbc:de43%58
   IPv4 Address. . . . . . . . . . . : 172.25.240.1
   Subnet Mask . . . . . . . . . . . : 255.255.240.0
   Default Gateway . . . . . . . . . :

Ethernet adapter VirtualBox Host-Only Network:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::6f8c:7df0:8963:38ef%8
   IPv4 Address. . . . . . . . . . . : 192.168.56.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

   Media State . . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

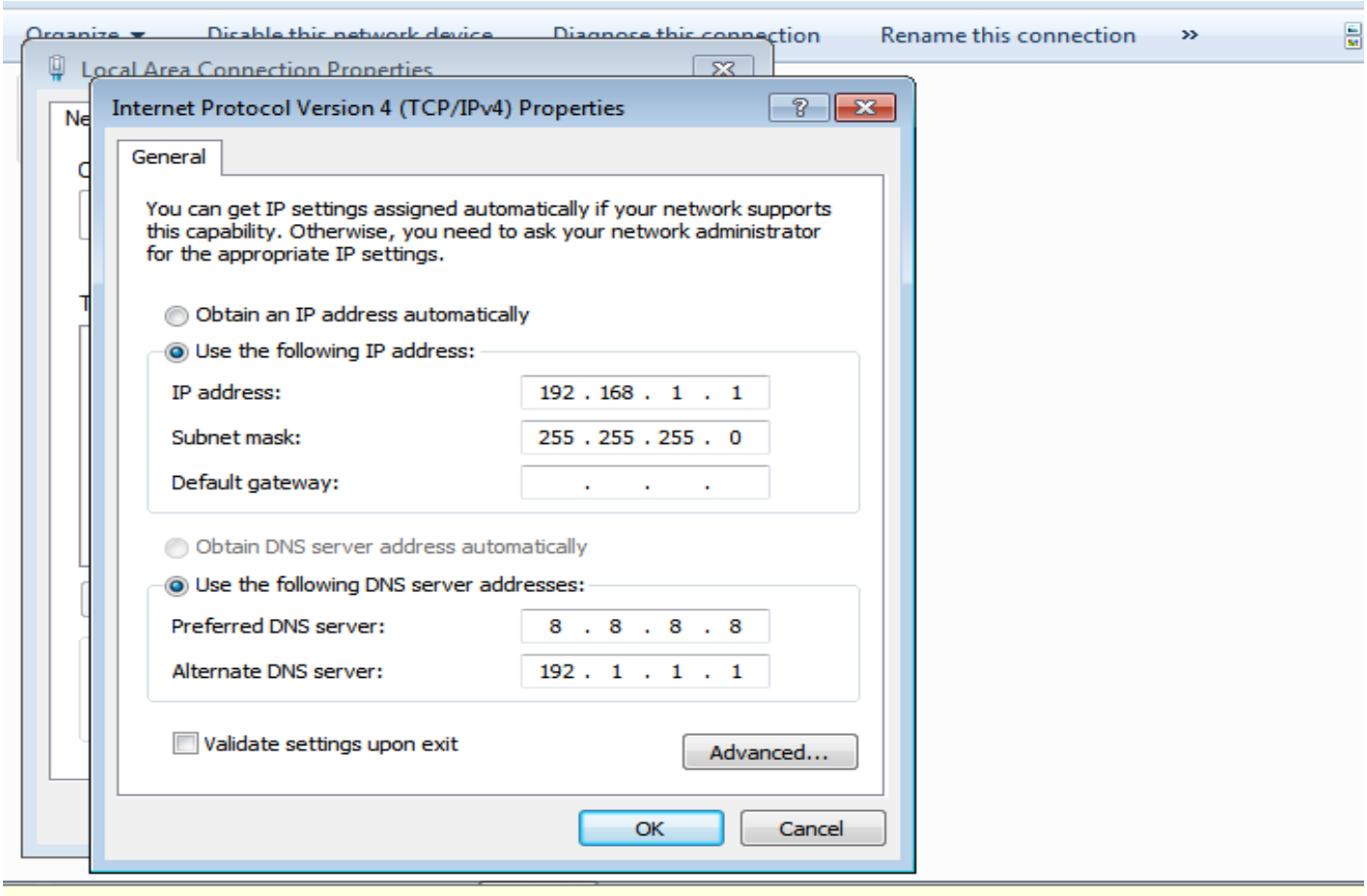
```
Ethernet adapter VMware Network Adapter VMnet1:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::f84b:b209:2ba7:963d%21
   IPv4 Address. . . . . . . . . . . : 192.168.109.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Ethernet adapter VMware Network Adapter VMnet8:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::8b0:1c80:db7f:d7d7%6
   IPv4 Address. . . . . . . . . . . : 192.168.117.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::902c:32d7:8933:12fd%13
   IPv4 Address. . . . . . . . . . . : 10.71.253.8
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.71.253.25

Ethernet adapter Bluetooth Network Connection:

   Media State . . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

Ifconfig Kali Linux output

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.117.128  netmask 255.255.255.0  broadcast 192.168.117.255
        inet6 fe80::84a9:d2d3:bf5f:5a19  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:11:31:ed  txqueuelen 1000  (Ethernet)
        RX packets 31184  bytes 45109100 (43.0 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 9479  bytes 579114 (565.5 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 8  bytes 480 (480.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 480 (480.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

- **TASK: Identify whether your IP is private or public and explain why.**





Based on the Windows ipconfig output.

IP address (192.168.1.1) is PRIVATE

Why is it private?

Because it falls within the **Class C private IP range**: `192.168.0.0 – 192.168.255.255`

Additional Observations:

Setting	Value	Analysis
IPv4 Address	192.168.1.1	✅ Private IP
Subnet Mask	255.255.255.0	Standard home/office network
Default Gateway	(blank)	Tis device might be the router itself

Above IP is private because:

1. It starts with 192.168 — reserved for private networks by IANA.
2. It cannot be routed on the public internet.
3. Any device with this IP exists only behind a NAT boundary.

Public IP (what the internet sees) would be whatever IP your ISP assigned to modem/router — completely different from this 192.168.1.1.

➢ TASK: IPv4 address structure (octets, binary, decimal)?

Here is the IPv4 address structure in simple text format.

IPv4 Address Structure (Simple Explanation).

An IPv4 address is a 32-bit number, but we write it as 4 smaller numbers separated by dots.

The 4 Octets

- The address is made of 4 parts called "octets" (because each part has 8 bits).
- Example:192.168.1.1
  - Octet 1: 192
  - Octet 2: 168
  - Octet 3: 1
  - Octet 4: 1

Decimal vs. Binary

- Computers read binary (0s and 1s), but we use decimal (0-9) for simplicity.
- Each octet in decimal ranges from 0 to 255.
- Each octet in binary is 8 digits long (e.g., 192 = `11000000).

Why 0 to 255?

- Because 8 bits can create 256 possible combinations (2^8 = 256).
- Counting starts at 0, so the max number is 255.

Full Binary Example

- Decimal: `192.168.1.1`
- Binary: `11000000.10101000.00000001.00000001`

Summary

- **Structure:** 4 numbers (0-255) separated by dots.
- **Total size:** 32 bits.
- **Purpose:** Every device on a network gets one to send and receive data.

➢ TASK: Address classes: Class A, B, and C ranges.

Here are the IPv4 address classes (A, B, and C) in simple text format.

Class A

Range: 1.0.0.0 to 126.255.255.255

First octet: 1 to 126.

Default subnet mask: 255.0.0.0 (or /8).

Network/Host: First octet = network | Last 3 octets = host.

Use: Very large networks (like big companies or ISPs).

Private range: 10.0.0.0 to 10.255.255.255.

Class B

Range: 128.0.0.0 to 191.255.255.255.

First octet: 128 to 191.

Default subnet mask: 255.255.0.0 (or /16).

Network/Host: First 2 octets = network | Last 2 octets = host.

Use: Medium to large networks (like universities).

Private range: 172.16.0.0 to 172.31.255.255.

Class C

Range: 192.0.0.0 to 223.255.255.255.

First octet: 192 to 223.

Default subnet mask: 255.255.255.0 (or /24).

Network/Host: First 3 octets = network | Last octet = host.

Use: Small networks (like home or small business).

Private range: 192.168.0.0 to 192.168.255.255.

Quick Way to Identify Class

- **Look at the first octet only:**
  - 1 to 126 = Class A
  - 128 to 191 = Class B
  - 192 to 223 = Class C

Special Note

- 127.x.x.x is reserved for loopback (localhost) — not used for regular networks.
- Class D (224-239) is for multicast.
- Class E (240-255) is reserved for experimental use.

Day 4 Tasks—Subnetting Basic

- **Solve 3 subnetting problems manually (show all working).**

Subnetting Procedure (Step-by-Step)

Follow this manual procedure to solve any sub netting problem:

The 5-Step Method

Step 1: Identify what you have

- IP address (e.g., 192.168.1.0)
- Subnet mask or CIDR (e.g., /24 or 2a55.255.255.0)

How many subnets or hosts needed?

Step 2: Find the mask in binary.

- Write the subnet mask in binary (1s = network, 0s = host)

Step 3: Calculate bits borrowed**

- For more subnets = borrow host bits (turn 0s into 1s)
- Formula: 2^n = number of subnets (n = bits borrowed)

Step 4: Calculate hosts per subnet

- Formula: 2^h - 2 = usable hosts (h = remaining host bits)

- Subtract 2 for network address and broadcast address

Step 5: Find subnet ranges (increment)

- Increment = 256 - last non-zero octet in mask
- List each subnet: network, first host, last host, broadcast

Problem 1: Basic Subnetting

Question: we have `192.168.1.0/24`. You need 4 subnets. Find the new mask, subnet ranges, and hosts per subnet.

Solution:

Step 1: Given IP = 192.168.1.0, Mask = /24 (255.255.255.0)

Step 2: Binary mask = `11111111.11111111.11111111.00000000`

Step 3: Need 4 subnets → 2^n = 4 → n = 2 bits to borrow

- **New mask:** Borrow 2 host bits
- **Binary:** 11111111.11111111.11111111.11000000
- **New CIDR**: /26

New subnet mask: 255.255.255.192

Step 4: Host bits left = 8 - 2 = 6 bits

Usable hosts = 2^6 - 2 = 64 - 2 = 62 hosts per subnet

Step 5: Increment = 256 - 192 = 64

Subnet ranges:

Subnet	Network	First Host	Last Host	Broadcast
1	192.168.1.0	192.168.1.1	192.168.1.62	192.168.1.63
2	192.168.1.64	192.168.1.65	192.168.1.126	192.168.1.127
3	192.168.1.128	192.168.1.129	192.168.1.190	192.168.1.191
4	192.168.1.192	192.168.1.193	192.168.1.254	192.168.1.255

Answer: New mask = /26 (255.255.255.192), 4 subnets, 62 hosts each

Problem 2: Subnet Based on Host Requirement

Question: we have `172.16.0.0/16`. You need subnets with at least 500 hosts each. Find the new mask and how many subnets you get.

Solution:

Step 1: Given IP = 172.16.0.0, Mask = /16 (255.255.0.0)

Step 2: Binary mask = `11111111.11111111.00000000.00000000`

- Host bits total = 16

Step 3: Need 500 hosts per subnet

- Formula: 2^h - 2 ≥ 500

- 2^9 = 512 - 2 = 510 hosts ✅ (500 fits)

- 2^8 = 256 - 2 = 254 hosts ❌ (too small)

- So h = 9 host bits needed

Step 4: Borrowed bits = Total host bits - Host bits needed

- Borrowed = 16 - 9 = 7 bits

New mask = /16 + 7 = /23

New subnet mask = 255.255.254.0

Step 5: Number of subnets = 2^7 = 128 subnets

Step 6: Increment = 256 - 254 = 2 (in the 3rd octet)

- Example subnets: 172.16.0.0/23, 172.16.2.0/23, 172.16.4.0/23...

Answer: New mask = /23 (255.255.254.0), 128 subnets, 510 hosts each

Problem 3: Find Subnet of a Specific IP

Question: IP address `10.50.100.75` with subnet mask `255.255.240.0`. What subnet is this IP in? What is the network address and broadcast address?

Solution:

Step 1: IP = 10.50.100.75, Mask = 255.255.240.0

Step 2: Convert mask to binary:

- 255.255.240.0 = 11111111.11111111.11110000.00000000

Step 3: Find which octet changes (the "interesting octet")

- First 2 octets are 255 → stay same as IP
- 3rd octet = 240 (interesting octet)
- 4th octet = 0 → all zeros in host portion

Step 4: Increment = 256 - 240 = 16 (in the 3rd octet)

Step 5: Find which subnet range contains 100 (3rd octet of IP)

- Subnets in 3rd octet: 0, 16, 32, 48, 64, 80, 96, 112, 128...
- 100 falls between 96 and 112
- Network 3rd octet = 96

Step 6: Build the network address

- Network = 10.50.96.0 (3rd octet = 96, 4th octet = 0)

Step 7: Find broadcast address

- Next network's 3rd octet = 112
- Broadcast = 10.50.111.255 (112 - 1 = 111 for 3rd octet, all 1s for host)

Step 8: Host range

- First host = 10.50.96.1
- Last host = 10.50.111.254

Answer:

- Network address: 10.50.96.0
- Broadcast address: 10.50.111.255
- This IP is in subnet: 10.50.96.0/20

Quick Reference Table

CIDR	Subnet Mask	Increment	Hosts
/24	255.255.255.0	1	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/29	255.255.255.248	8	6
/30	255.255.255.252	4	2

How to Perform This Practically (Hands-on)

What you need:

- Paper and pen (or notepad)
- Calculator (optional)

Step-by-step practical exercise:

1. Write down the problem** exactly as given
2. Draw a table with 4 columns: Subnet # | Network | First Host | Last Host | Broadcast.
3. Convert to binary (practice writing 8 bits for each octet)
4. Calculate increment (256 - interesting octet value)
5. List subnets by adding increment repeatedly
6. Verify your work:
   - Network address = all host bits = 0
   - Broadcast = all host bits = 1
   - Hosts between them

➢ Task: Calculate Subnet Masks for Given Requirements?

The Basic Formula

Two types of requirements:

1. Number of subnets needed→ Borrow bits from host portion.
2. Number of hosts per subnet needed → keep enough host bits.

Procedure (Simple 4 Steps)

Step 1: Start with the default mask based on IP class

- **Class A**: /8 (255.0.0.0)
- **Class B:** /16 (255.255.0.0)
- **Class C:** /24 (255.255.255.0)

Step 2: Determine how many bits to borrow or keep

- **For subnets: 2^n ≥ number of subnets needed**
- **For hosts: 2^h - 2 ≥ number of hosts needed**

Step 3: Calculate the new mask

- New CIDR = Default CIDR + n (bits borrowed)

- Or New CIDR = 32 - h (host bits kept)

Step 4: Convert CIDR to dotted decimal

- Write binary: n ones, then zeros
- Convert each 8-bit group to decimal

Example 1: Need 8 Subnets from Class C

Requirement: 8 subnets from 192.168.1.0

Solution:

- Default mask = /24

- 2^n ≥ 8 → n = 3 bits borrowed
- New CIDR = 24 + 3 = /27
- Binary mask: 11111111.11111111.11111111.11100000
- Dotted decimal: 255.255.255.224

Answer: 255.255.255.224

Quick Reference Table

Requirement	Default	Calculation	New Mask
2 subnets from Class C	/24	2^1 → /25	255.255.255.128
4 subnets from Class C	/24	2^2 → /26	255.255.255.192
8 subnets from Class C	/24	2^3 → /27	255.255.255.224
16 subnets from Class C	/24	2^4 → /28	255.255.255.240
32 subnets from Class C	/24	2^5 → /29	255.255.255.248
126 hosts from Class C	/24	2^7-2 → /25	255.255.255.128
62 hosts from Class C	/24	2^6-2 → /26	255.255.255.192
30 hosts from Class C	/24	2^5-2 → /27	255.255.255.224
14 hosts from Class C	/24	2^4-2 → /28	255.255.255.240

Common Mask Values (Dotted Decimal)

CIDR	Mask	Binary (last octet)
/24	255.255.255.0	00000000
/25	255.255.255.128	10000000
/26	255.255.255.192	11000000
/27	255.255.255.224	11100000
/28	255.255.255.240	11110000
/29	255.255.255.248	11111000
/30	255.255.255.252	11111100

CIDR	Mask	Binary (2nd & 3rd octet)
/8	255.0.0.0	11111111.00000000.00000000
/9	255.128.0.0	11111111.10000000.00000000
/10	255.192.0.0	11111111.11000000.00000000
/11	255.224.0.0	11111111.11100000.00000000
/12	255.240.0.0	11111111.11110000.00000000
/13	255.248.0.0	11111111.11111000.00000000
/14	255.252.0.0	11111111.11111100.00000000
/15	255.254.0.0	11111111.11111110.00000000
/16	255.255.0.0	11111111.11111111.00000000

Quick Reference Table

CIDR	Subnet Mask	Total IPs	Usable IPs	Common Use
/8	255.0.0.0	16,777,216	16,777,214	Large organizations
/16	255.255.0.0	65,536	65,534	Medium networks
/24	255.255.255.0	256	254	Small LAN (standard)
/28	255.255.255.240	16	14	Small subnets
/30	255.255.255.252	4	2	Point-to-point links
/32	255.255.255.255	1	1	Single host (loopback)

➢ Task :CIDR notation (e.g. /24, /28)

CIDR:

/24 = 255.255.255.0

/28 = 255.255.255.240

Network bits = fixed portion

Host bits = variable portion

CIDR (Classless Inter Domain Routing) is a method of IP address allocation and routing that allows more efficient use of IP addresses. Unlike traditional class-based addressing, CIDR allocates IP addresses based on a network prefix rather than a fixed class (A, B, or C).

CIDR Notation: a.b.c.d/n

n = number of bits in the network prefix.

Example: 192.168.1.0/24 first 24 bits are the network, and the remaining 8 bits are the host ID.

Why CIDR?

Classful addressing wastes IP addresses:

Class	IPs Available	Hosts	Example Wastage
A	2^{24}	$2^{24} - 2$	Too large for small orgs
B	2^{16}	$2^{16} - 2$	Wastes 49,150 hosts for 214 needed
C	2^{8}	$2^{8} - 2$	Small networks only

Problem: Organizations often need a number of hosts that do not match class sizes, leading to wastage.
Solution: CIDR allows flexible block allocation matching exact requirements.

Rules for Forming CIDR Blocks

- All IPs must be contiguous.
- Block size must be a power of 2 (2^n) simplifies network division.
- First IP of block divisible by block size least significant bits of host ID should be 0. 1/6 5

Example: If the Block size is 2 then, Host Id will contain 5 bits and Network will contain 32 - 5 = 27 bits. First IP address of the Block must be evenly divisible by the size of the block. In simple words, the least significant part should always start with zeroes in Host Id. Since all the least significant bits of Host Id is zero, then we can use it as Block Id part.

Example: Check whether 100.1.2.32 to 100.1.2.47 is a valid IP address block or not? 4

- All the IP addresses are contiguous.
- Total number of IP addresses in the Block = 16 = 2.
- 1st IP address: 100.1.2.00100000 Since, Host Id will contains last 4 bits and all the least significant 4 bits are zero. Hence, first IP address is evenly divisible by the size of the block.

All three rules are followed by this Block. Hence, it is a valid IP address block.

Advantages of CIDR

- **Efficient IP usage:** Minimizes IPv4 address wastage.
- **Flexible allocation:** Supports networks of any size.
- **Improved routing:** Aggregates addresses for faster, simpler routing.
- **Lower administrative overhead:** Easier IP and network management.

Disadvantages of CIDR

- **Complexity:** CIDR is more complex to implement and manage compared to traditional class-based addressing. Compatibility
- **Issues:** Some older network devices may not support CIDR.
- **Security Concerns:** Implementing security measures like firewall rules and access control lists can be more difficult.

CIDR was introduced primarily to

A

Increase the number of IPv4 addresses

B

Replace classful addressing and reduce IP address wastage

C

2/6 Make routing tables smaller forever

D

Support IPv6 only

In CIDR notation, 192.168.10.0/25 means

A

25 host bits, 7 network bits

B

25 network bits, 7 host bits

C

25 total bits are used

D

25 subnets are possible

What is the block size (number of IP addresses) for a /28 network?

A	**8**
B	**14**
C	**16**
D	**32**

Before CIDR, IP addresses used fixed classes (A, B, C), which wasted many addresses. CIDR allows flexible subnet size you can have a /27 (32 IPs) or /29 (8 IPs) instead of only /24 (256 IPs) or /16 (65k IPs).

➢ TASK: Difference between Network bits and Host bits.

Initially in 1980 IP address was divided into two fixed parts NID=8 bit and HID=24 bit. NID (Network ID) represents the entire network for a specified host. HID (Host ID) represents a specified user IP address within the network. So there are only 2 i.e., 256 networks are available and there are 2 i.e., 16M hosts per network. If even a small organization wants to purchase a network then it should buy one network in which there are 16M hosts that can be connected this is the drawback of this fixed representation of IP address. 8 IP addresses

are divided into 5 classes: Class A is used for big networks such as ISP networks. Class B is used by big organizations. Class C is used by small organizations. Class D no NID and HID all 28 bits are used to define the multicast address. Class E is reserved for research and future purchases.

Range of All Classes

Class A: 1.0.0.0 to 127.255.255.255

Class B: 128.0.0.0 to 191.255.255.255

Class C: 192.0.0.0 to 223.255.255.255

Class D: 224.0.0.0 to 239.255.255.255

Class E: 240.0.0.0 to 255.255.255.255

What is Network ID?

A network ID also known as NetID is a fixed bit in the IP address that represents the entire network of any host it is connected to the network. i.e., it tells the network of the host by which the host is connected. The total size of the IP address is 32 bits and the IP address is divided into four octets each of size is 8 bits. And further, divide into NID and HID. Let's discuss each class in detail.

Class A: The first octet represents NID so there are 8 bits for NID and 24 bits for HID. And the first bit of the first octet is always 0.

Class B: The first two octets represent NID so there are 16 bits for NID and 16 bits for HID. And the first two bits of the first octet are fixed i.e., 10.

Class C: The first three octets represent NID so there are 24 bits for NID and 8 bits for HID. The first three bits of the first octet are fixed i.e., 110.

Class D: All bits are fixed for NID.

Class E: All bits are fixed for NID.

The network ID uses a prefix based on its class. Classes A, B, C, D, and E use prefixes '0', '10', '110', '1110', and '1111 0', respectively. Additionally, as seen in the graphic below, each class has a different number of octets reserved for the host ID. Class A reserves 3 octets, class B reserves 2, class C reserves 1, and class D/E have no reserved octets for the host ID. Combining these two facts, we see that Classes A, B, C, D, and E use 7, 14, 21, 28, and 27 bits, respectively, for the actual network ID.

What is Host ID?

It is the fragment of an IP address that uniquely identifies a host within a network. The network ID can be found by ANDing the IP (in binary) and subnet mask (in binary). The host ID is found in the same way, except one must first invert the subnet mask (not an official term, but could that binary value the 'host mask').

Question 1: What is the network address if the destination address is 200.45.34.56 and the subnet mask is 255.255.255.240?

Solution: We know the range of class C is from 192 to 223 so the given IP address belongs to class C. As given subnet mask 255.255.255.240 To find the network address we perform ANDing operation between the

IP address and subnet mask. 200.45.34.00111000 AND 255.255.255.11110000 we get 200.45.34.48 as a network address.

Question 2: A company is granted the site address 181.56.0.0. The company needs 1000 subnets. Find the subnet mask

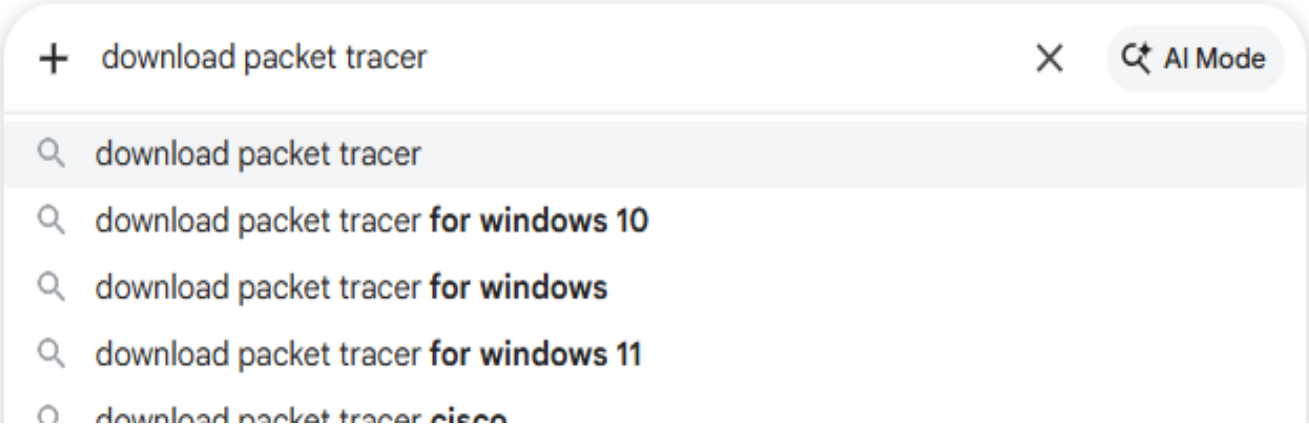
Solution: As we know the range of class B IP addresses in the first octet is from 128 to 191 so the given address belongs to class B. and the default subnet mask of class B is, 255.255.0.0 to get 1000 subnets we perform a log operation log 1000=10 So, we require extra 10 bits from 255.255.00000000.00000000 to get the subnet mask i.e.; 255.255.00000000.000000000 So, the required subnet mask is 255.255.11111111.11000000 or 255.255.255.192.

Conclusion

ID address is divided into two parts that is Network ID and Host ID. Network ID indicates the specific network to which the address belongs. Host ID Identifies the specific device within that network. Together, these components enable efficient routing of data packets across the networks, ensuring proper delivery to the correct device.

Day 5 Tasks—Install packet tracer

- **TASK: Download and install Cisco Packet Tracer?**



Supermarket Simulator
Your retail adventure begins Safe & fast download link

Hide sponsored result

Cisco Networking Academy: Learn Cybersecurity, Python & More
https://www.netacad.com › articles › news › download-...

Level up your skills with Cisco Packet Tracer 9.0.0

Industrial Cybersecurity Essentials · Download Cisco Packet Tracer 9.0.0 today and experience the latest in hands-on network simulation! Read more

Fileion.Com
https://cisco-packet-tracer.fileion.com › ... › Windows

Cisco Packet Tracer Download v9.0.0 for Windows 11/10

02-Mar-2026 — Downloading Cisco Packet Tracer 9.0.0 for Windows from Fileion. 100% Safe and Secure ✓ Free Download (32-bit/64-bit) Latest Version 2026.

4.5 ★★★★★ (6,340) · Free · Windows · Communication

Software Downloads

Search for infinite possibilities....

Apps Games Articles Sign Up

Advertisement

Continue to download

Continue to download

Chat Smarter

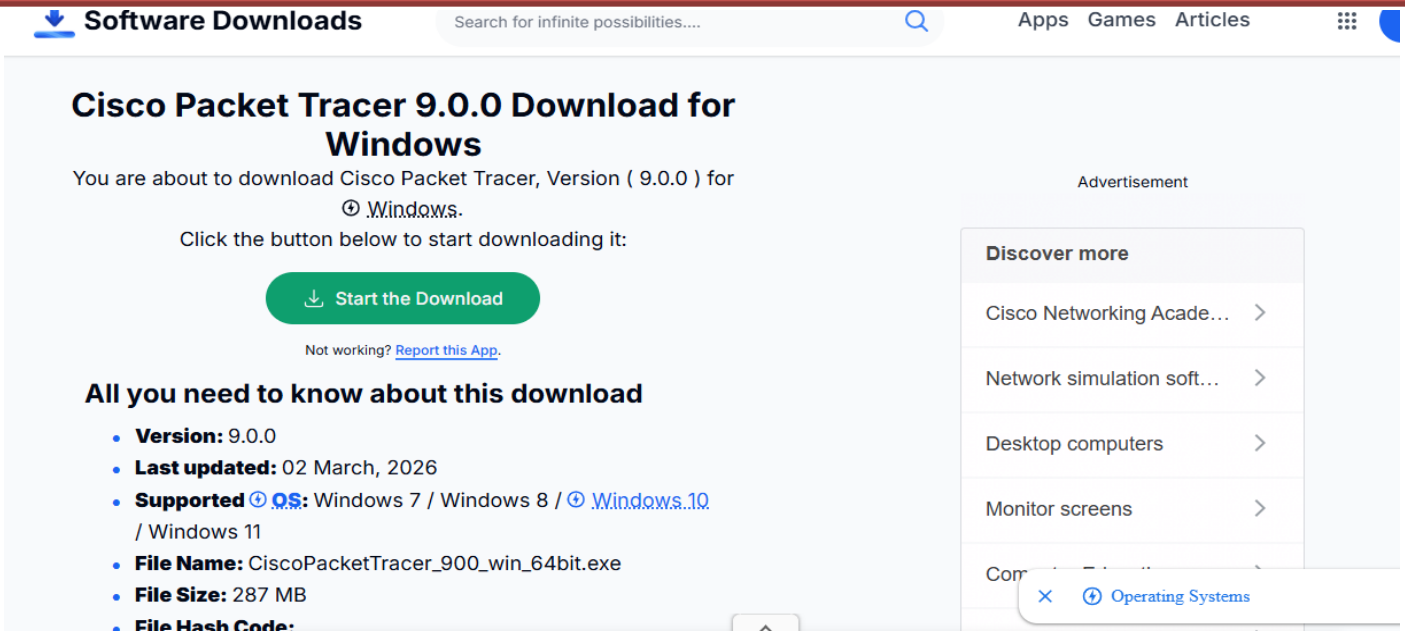
Cisco Packet Tracer

Home Features FAQs Versions

Check out other versions of Cisco Packet Tracer for Windows

Version	Compatibility	Language	Size	Last Updated
9.0.0	Windows 7 / Windows 8 / Windows 10 / Windows 11	English	287 MB	29, November 2025
8.2.2	Windows 7 / Windows 8 / Windows 10 / Windows 11	English	228 MB	29, August 2022
8.2.1	Windows 7.0 / Windows 8.1 / Windows 10 / Windows 11	English	227 MB	22, August 2022
8.2.0	Windows 7.0 / Windows 8.1 / Windows 10 / Windows 11	English	223 MB	18, August 2022
8.1.1	Windows 7.0 / Windows 8.1 / Windows 10 / Windows 11	English	222 MB	18, March 2022

Routers



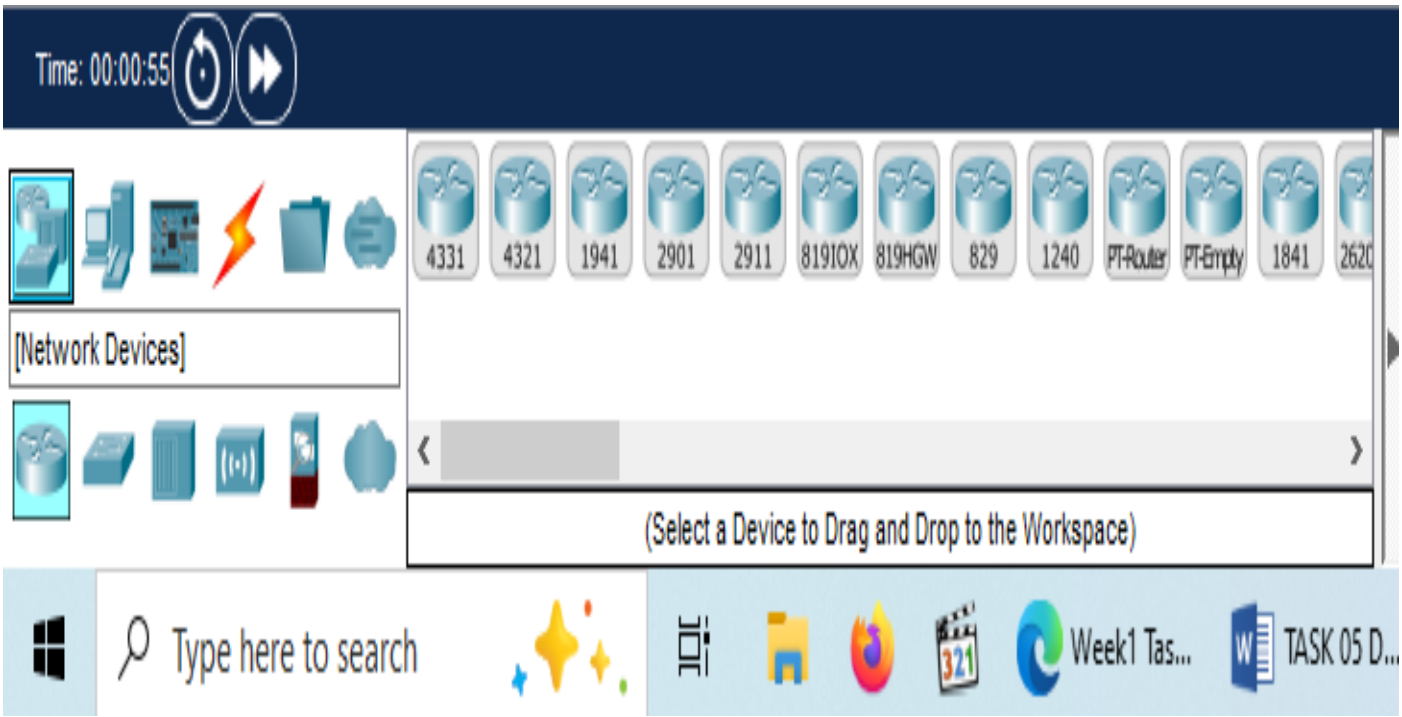
Install Packet Tracer

Steps:

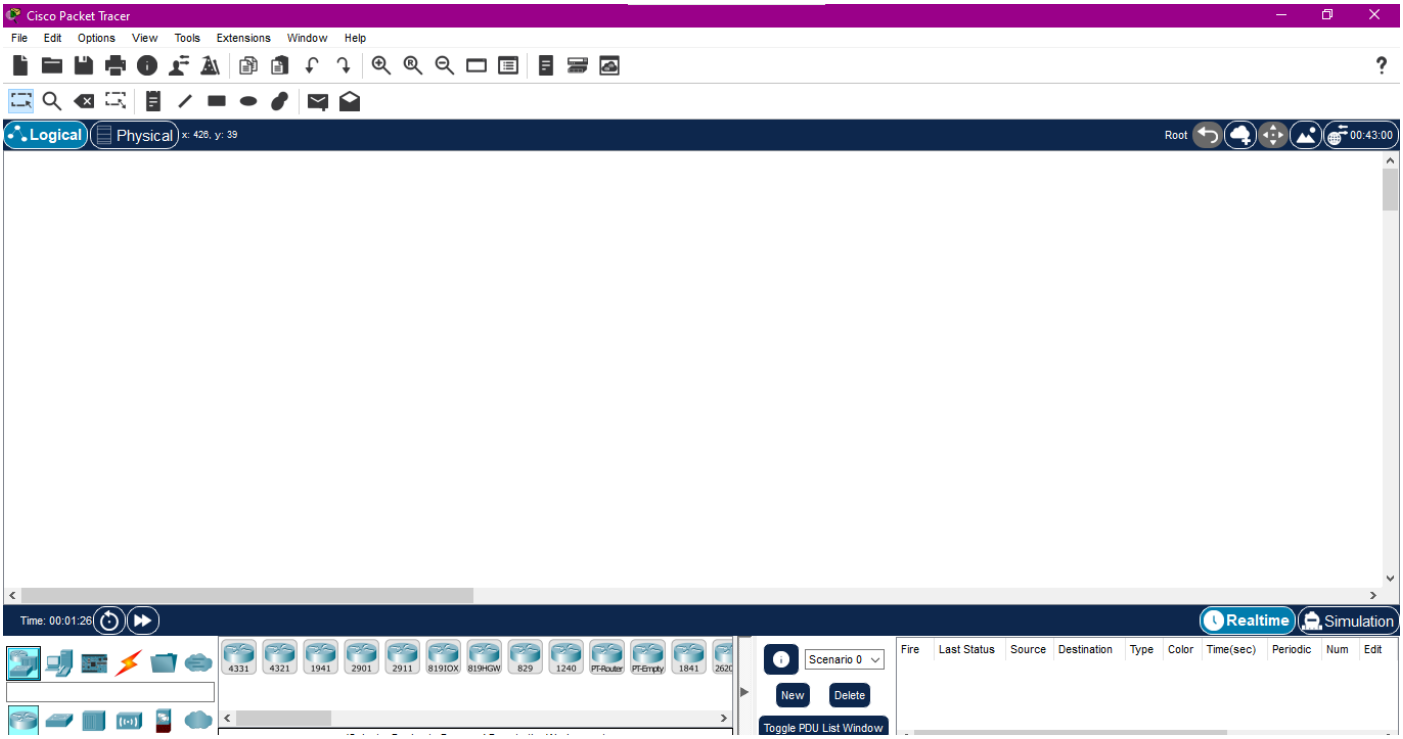
1. Download from Cisco NetAcad (free registration)
2. Install on Windows/Linux/Mac.

➢ TASK: Explore the interface: device panel, workspace, CLI?

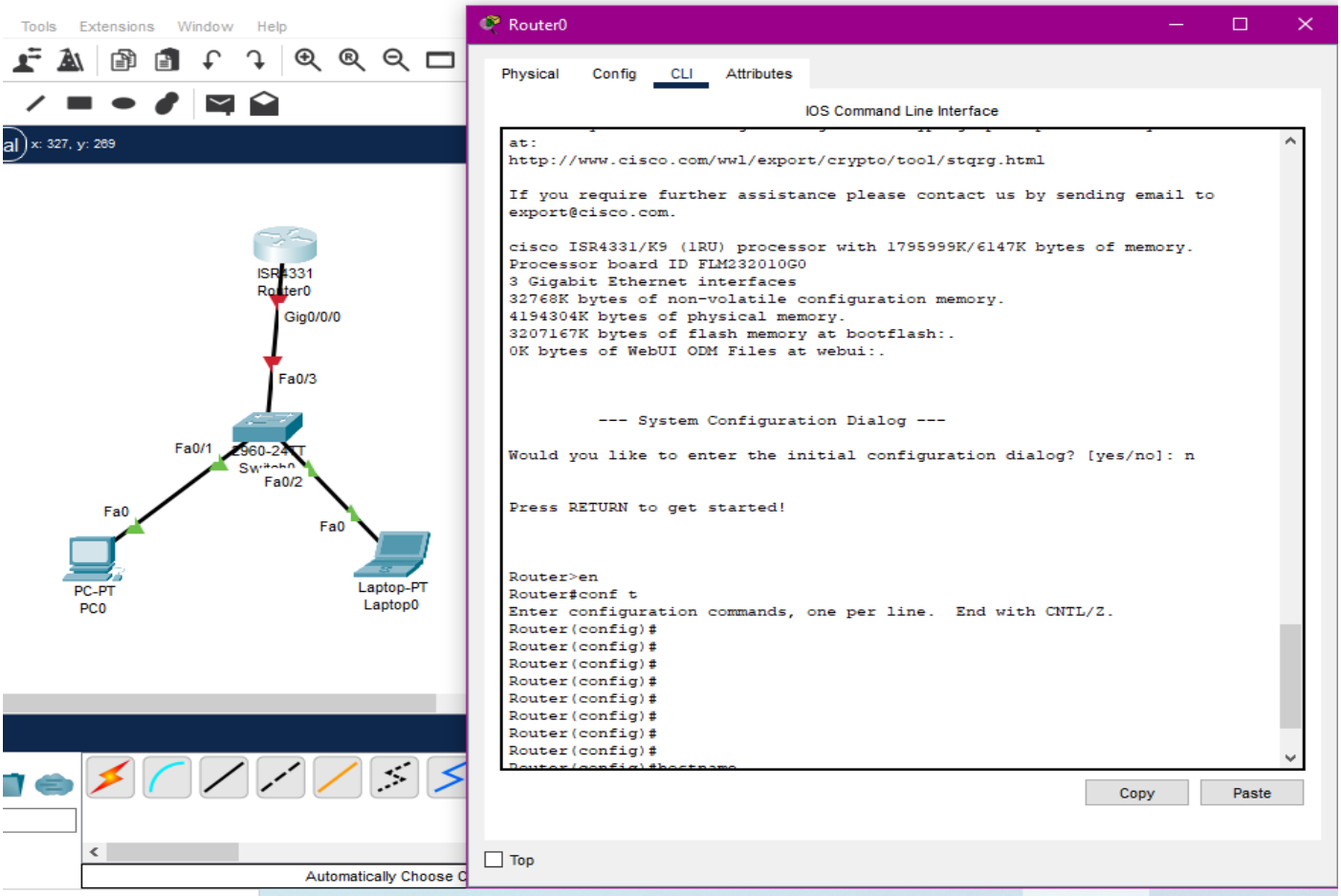
Device panel



Workspace



CLI



➢ TASK: Purpose and benefits of network simulation tools?

Virtualization revolutionized IT networking, making switches and routers almost as powerful as servers. It facilitates running containers and virtual machines directly on switches to maximize hardware utilization, streamline operations and enhance security.

Meanwhile, network simulation tools help IT engineers working in virtualized environments by providing a cost-effective and risk-free way to design, test and optimize networks before deployment in a production environment. They even help boost cybersecurity by enabling engineers to simulate threats, intrusion attempts, and disaster recovery scenarios to enhance network security and resilience.

So whether you work at a small business looking to optimize resources or a large enterprise seeking agility, virtualization offers a flexible solution to meet evolving IT demands. With the addition of network simulation platforms, even the smallest shops get change control to determine the behavior a networking alteration may have before it goes live.

Common Network Simulation Platforms

In this blog, we'll explore the benefits and drawbacks of some notable platforms that facilitate network design, experimentation and performance analysis. They include:

- **Eve-NG:** Known for its flexibility and support for a wide range of network devices.
- **Container Lab:** Offers container-based network labs, making it lightweight and efficient.
- **Cisco Modeling Labs (CML):** Provides a robust environment for simulating Cisco networks.
- **GNS3:** A well-known and user-friendly platform that supports a variety of network devices.

The Adaptability of Network Simulation Tools

Whether you're repurposing older hardware, leveraging newer systems, or deploying within existing virtualized environments, these network simulation platforms can be adapted to meet a wide range of needs. Here are a few additional points to consider about how each of these systems can integrate with physical hardware and virtualized environments.

Repurposing Older or Newer Hardware

Eve-NG, GNS3, Cisco Modeling Lab, and Container Lab can all run on various hardware configurations, from older machines with sufficient RAM and CPU resources to modern, high-performance systems. This makes them cost-effective solutions for many network engineers who want to set up their own lab environments.

Older Hardware: For example, a system with 16GB+ of RAM and a decent multi core processor can easily run multiple virtual devices using these platforms. If the hardware is only going to be used for light or small-scale labs, even a modest machine can work well.

Newer Hardware: More powerful systems can handle more complex topologies, advanced routing protocols, or virtualized instances of different network appliances. Additionally, with hardware advancements like faster SSDs and more cores, network engineers can emulate large-scale environments much faster and more efficiently.

Virtualization Support: Some older processors or motherboards may lack support for hardware virtualization, limiting the types of virtual machines that may be run.

Deployment in Existing Virtualized Environments

These platforms are often used in conjunction with virtualization technologies like VMware, Hyper-V, or KVM, enabling network engineers to deploy them in environments that are already virtualized. **VMware/Hyper-V:** Engineers can run Eve-NG, Cisco Modeling Lab, or GNS3 as virtual machines themselves on a hypervisor, utilizing the existing virtualized infrastructure. This approach allows for easier resource management, better scalability, and more efficient use of physical resources.

KVM/Proxmox: These platforms can also be deployed within a KVM or Proxmox virtualized environment. In fact, some network engineers prefer Proxmox for its flexibility and ability to create containerized environments or virtual machines for lab setups.

Integration with Existing Hardware

Physical Device Integration: One of the key advantages of network virtualization is the ability to integrate virtual networks with actual hardware devices, making it more than just a virtualized setup. Both Eve-NG and GNS3, for example, allow you to connect virtual devices to physical devices like routers, switches, firewalls, or even wireless APs and phone systems.

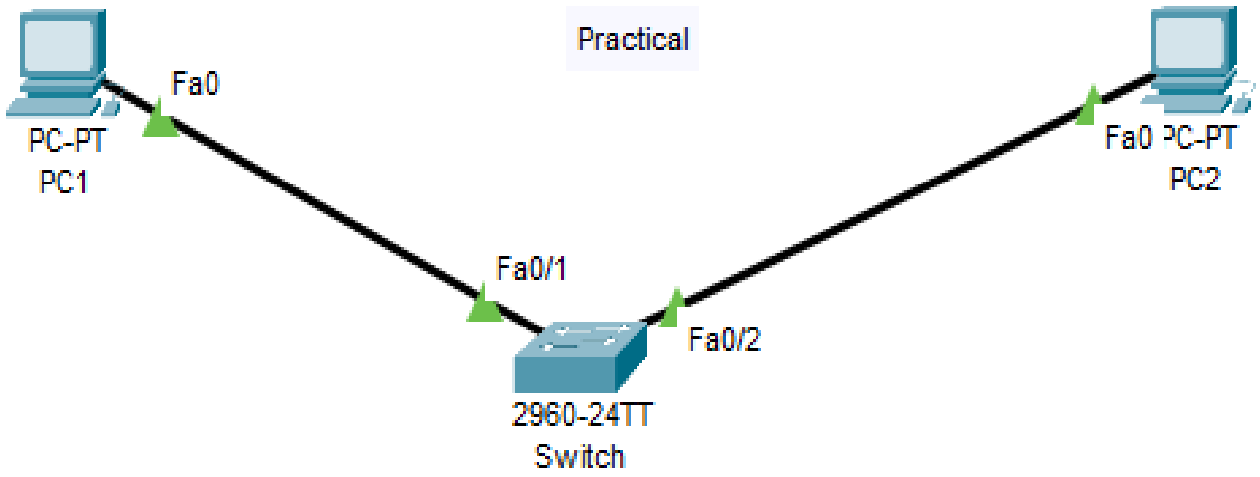
For Example:

- **GNS3:** It supports integration with physical devices using tools like the GNS3 VM, which can connect virtual machines with real hardware via Ethernet interfaces. This allows you to create hybrid environments where virtual and physical devices work seamlessly together. You can even run network cables to real physical routers or switches and connect them to virtual routers or firewalls, enabling end-to-end testing.
- **Eve-NG:** Like GNS3, Eve-NG allows for connecting virtual devices to physical hardware, which can be useful when you want to simulate a live network environment and ensure configurations work on real devices before pushing them into production.

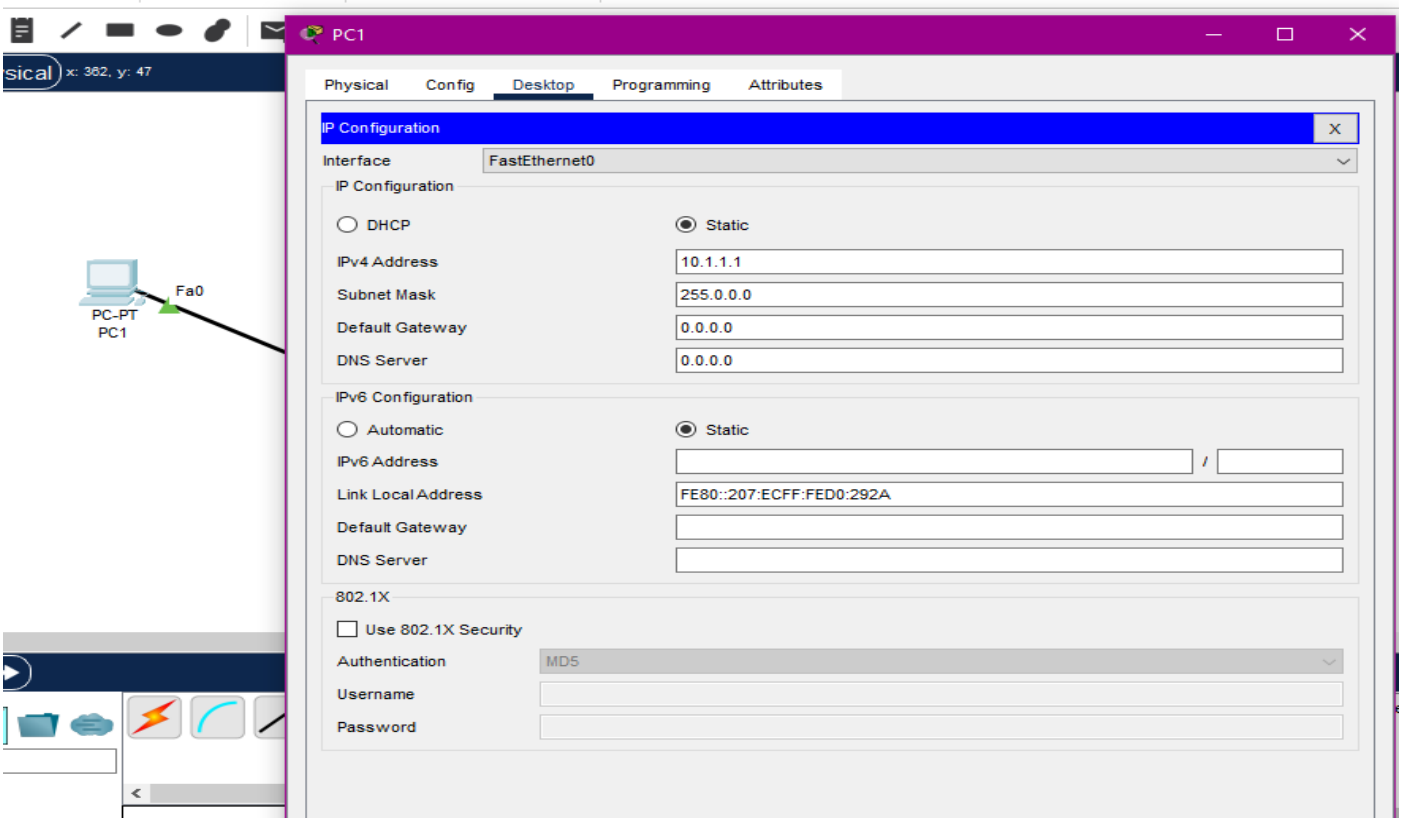
Day 6 Tasks—Simple Network Topology

Practical:

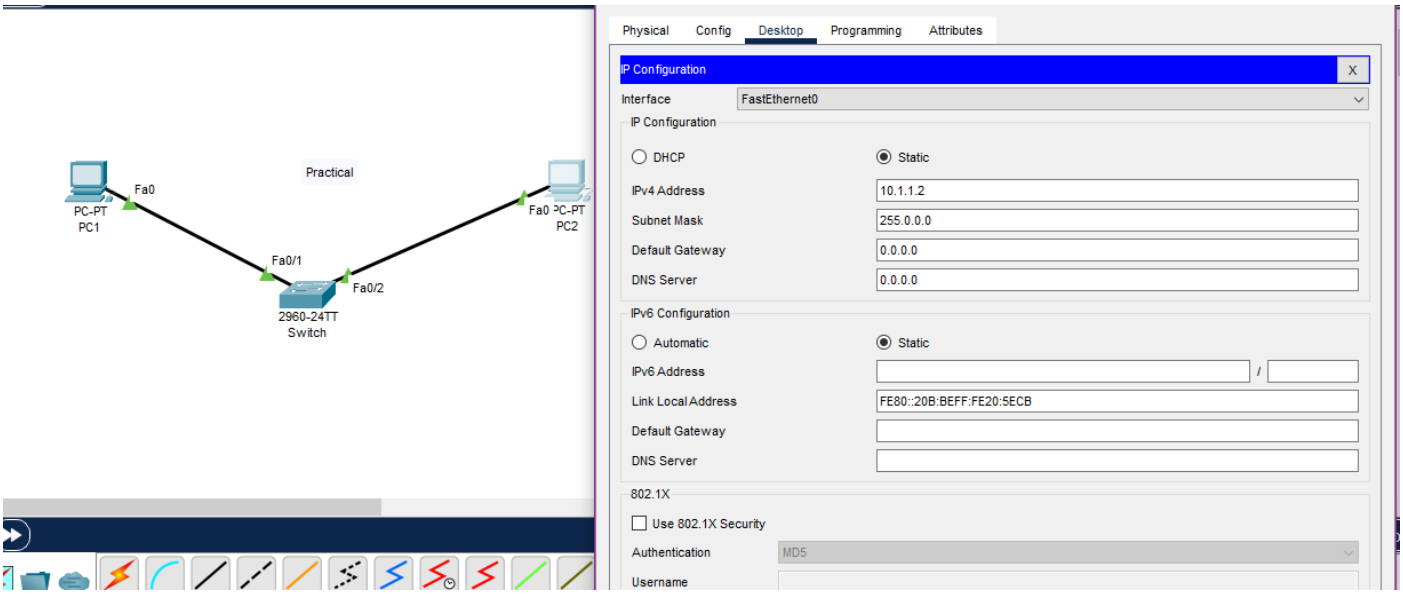
- **Build in Packet Tracer: 2 PCs connected via 1 Switch.**



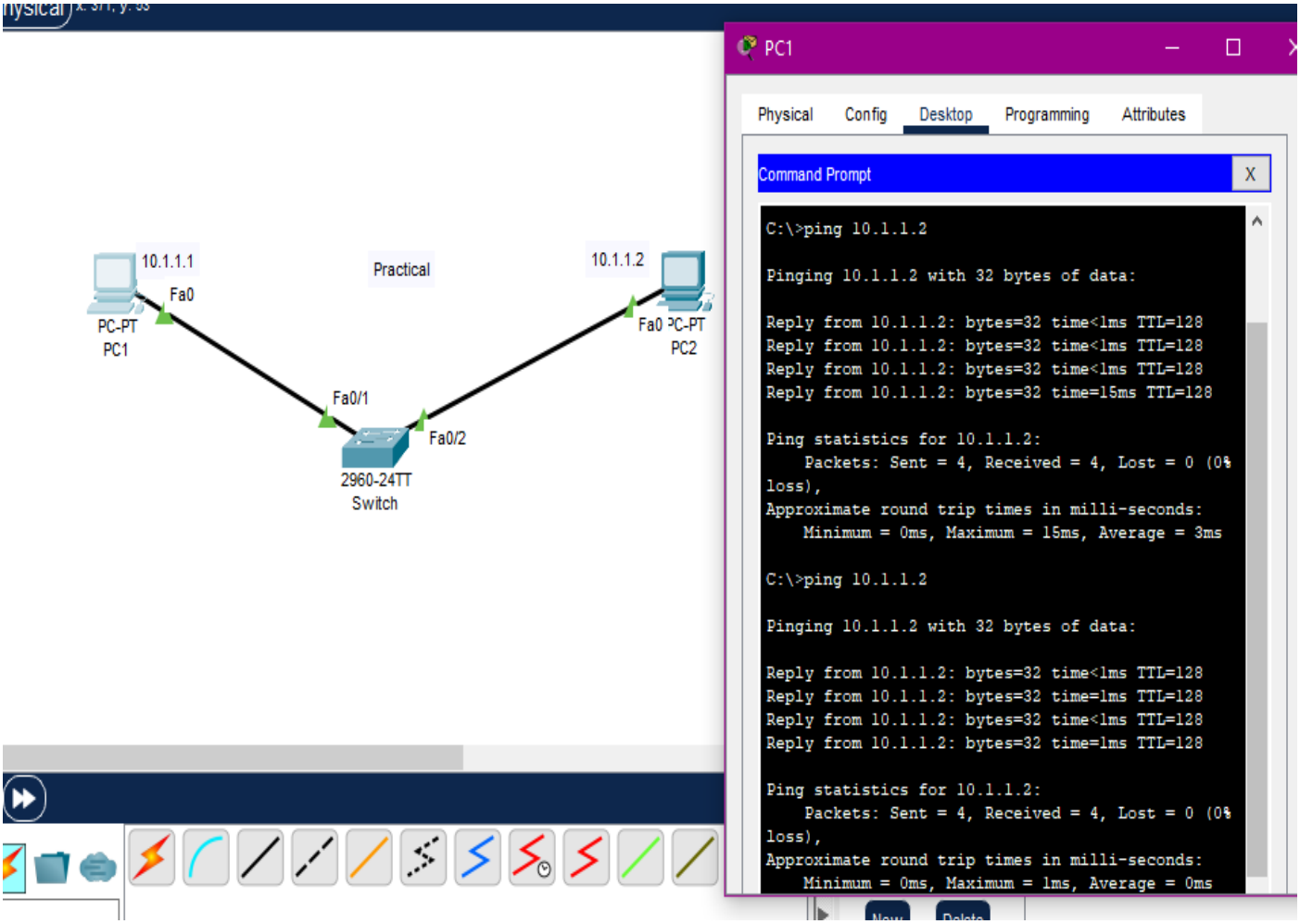
- **Task:Assign appropriate IP addresses to both PCs**



➢ PC 2 IP address Assign:



➢ TASK:Verify connectivity with a ping test.



➢ TASK: Star topology: structure, advantages, and use cases.

What is Star Topology?

Star Topology is a network setup in which each device is connected to a central node called a hub. The hub manages the data flow between the devices. If one device wants to send data to another device, it has to first send the information to the hub, and then the hub transmits that data to the required device. A star may be a Local Area Network (LAN) topology where all nodes are individually connected to a central connection point (hub). The number of links required to connect nodes in the star topology is N where N is the number of nodes. The hub or switch manages and controls all functions of the network.

How Does Star Topology Works in Computer Networks?

As we know, all the nodes in a star topology are connected to the Hub, and the Hub is responsible for the transmission of the data in the network. Whenever any node wants to transmit data to another node, it first transmits data to the central node (hub), and then it transfers the data to all the nodes on the network. After receiving the data, the hub checks for the destination address, and if the address matches the data is accepted otherwise data is rejected.

Types of Star Topology

There are two types of star topology:

1. Active Star Topology
2. Passive Star topology

- **Active Star Topology:** In active star topology, the central hub regenerates the signal when it passes through it. This simply means that the hub not only works as a connector but also boosts the signal and makes sure the signals reach their destination properly.
- **Passive Star Topology:** In passive star topology, the central hub simply connects the devices, but does not regenerate the signals. Here, the devices connected are responsible for the transmission of signals. Passive star topology is recommended for smaller setups.

What are the best practices of Star Topology?

Below mentioned are some of the applications of Star Topology:

- **Home Networks:** Star Topology is used in a home network where all devices connects to a connected to a central router that manages the Internet.
- **ATM Networks:** In banks, ATMs are also connected in as a star network, as it helps in managing data efficiently.
- **Hospital Networks:** Hospital devices, medicinal equipment are connected in the star topology. It helps in easily accessing data records.
- **CCTV Networks:** Star Topology are also used in CCTV Networks where multiple cameras are connected to central video recorder.

What is the Importance of Star Topology in LAN?

Star Topology is important in Local Area Network because it helps **in easy management and scalability** of the network. As central hub is present in the star topology, it becomes simple to monitor and control network traffic among devices present in the network. Because of star topology, adding or removing devices becomes easy and it also helps in expanding networks.

Advantages of Star Topology

- It is very reliable as if one cable or device fails then all the others will still work.
- It is high performing as no data collisions can occur.
- It is less expensive because each device only needs one I/O port and wishes to be connected to the hub with one link.
- Easy fault detection because the links are often easily identified.

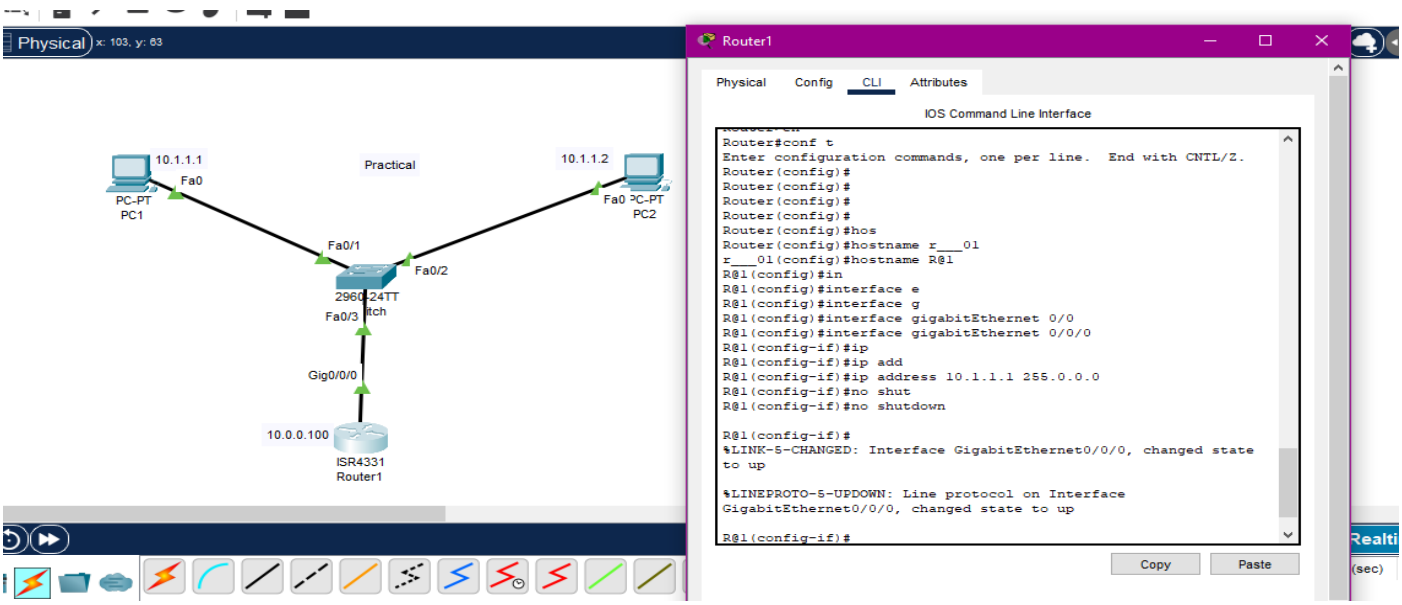
Disadvantages of Star Topology

- Requires more cable than bus topology.
- If the connecting network device (network switch) fails, the nodes attached are disabled and can't participate in network communication.
- More expensive than linear bus topology due to the value of the connecting devices (network switches).
- If the hub goes down everything goes down, none of the devices can work without the hub.
- Hub requires more resources and regular maintenance because it's the central system of Star.

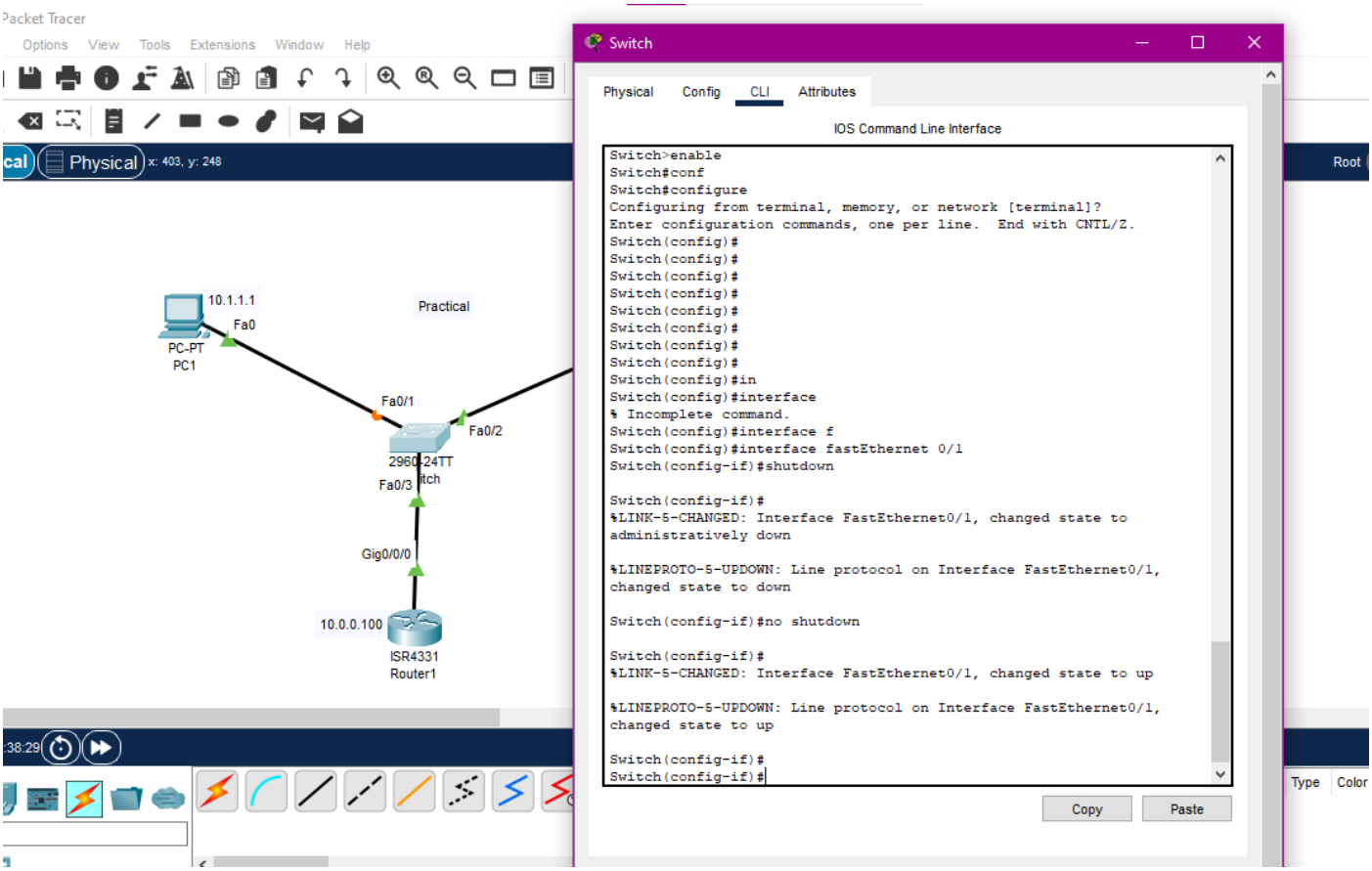
Day 7 Tasks—Router and Switch Configuration

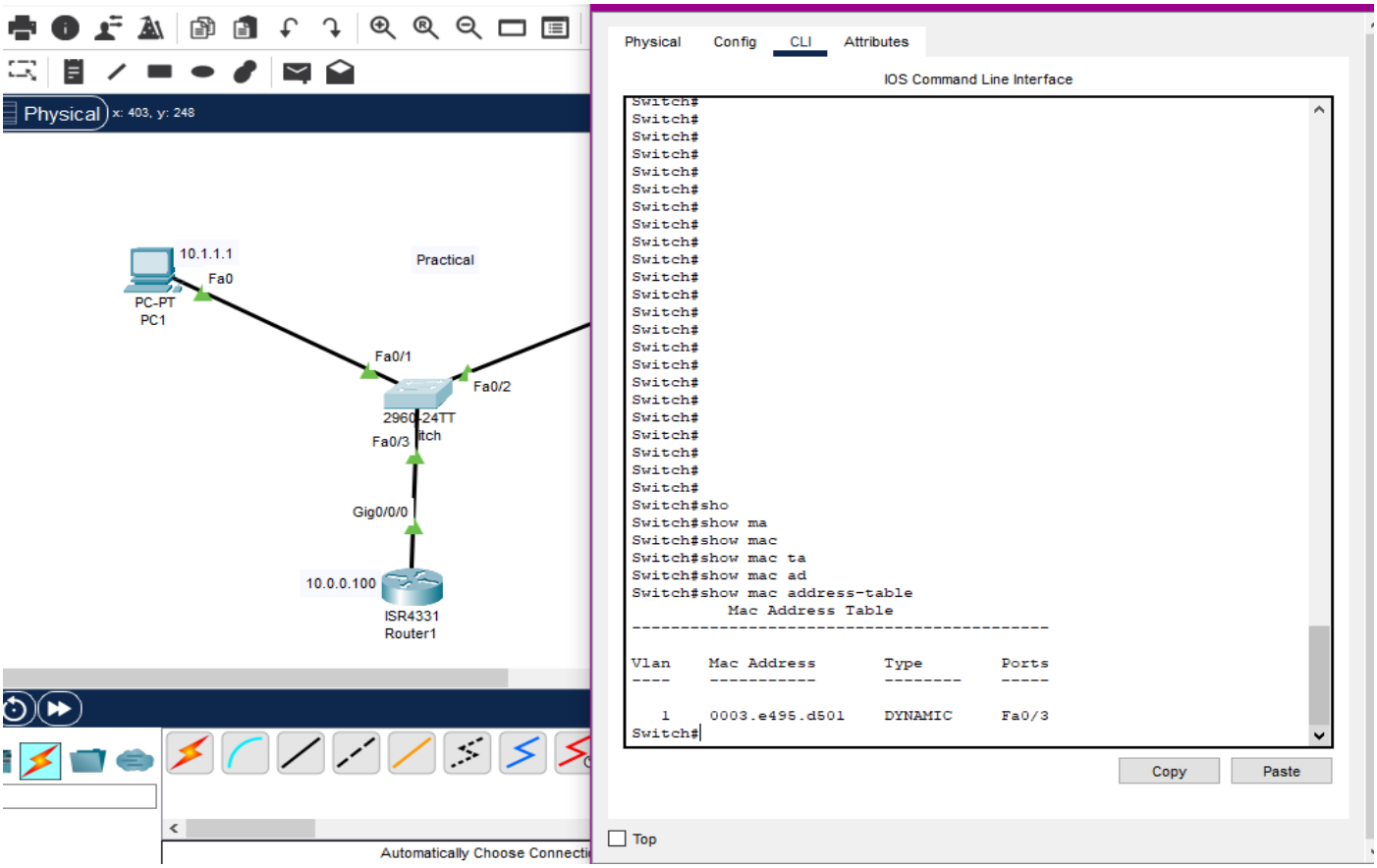
Practical:

Configure router IP address via CLI.



➢ Enable and disable switch ports; verify the MAC address table.





➢ TASK: Role of a router in a network.

A router is a networking device that forwards data packets between different computer networks. It connects multiple packet-switched networks or subnetworks, managing traffic by directing packets to their intended IP addresses. Routers allow multiple devices to share an Internet connection efficiently.

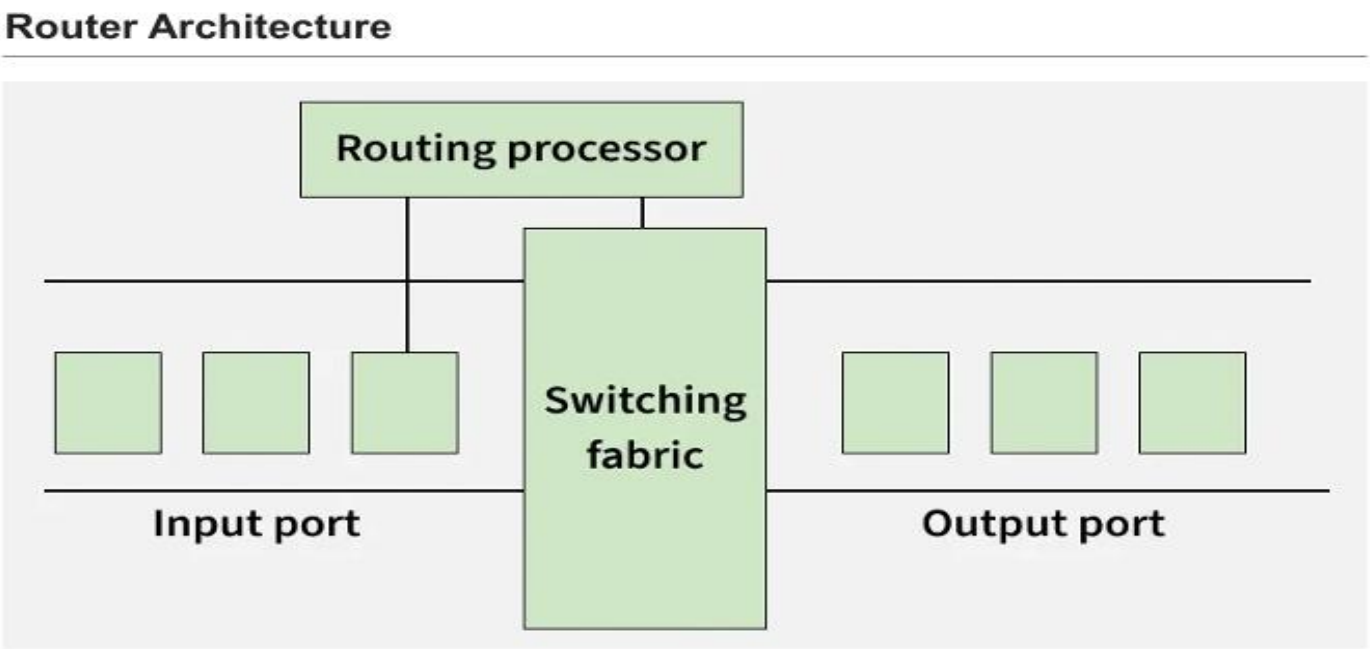
How Does a Router Work?

Routers determine the path for a packet by examining its destination IP address and consulting the routing table, which contains information on network paths. They use a set of rules to identify the most efficient route for each packet.

- **Static routing:** Configured manually, suitable for small or stable networks.
- **Dynamic routing:** Automatically updated based on network activity, ideal for large or changing networks.

Functions of a Router

- **Forwarding:** Receives packets, examines headers, and forwards them to the correct output port.
- **Routing:** Determines the optimal path for packets using routing tables and algorithms.
- **Network Address Translation (NAT):** Translates private IPs to a public IP for Internet access.
- **Security: Supports** firewalls and other security measures.
- **VPN Connectivity:** Provides secure remote access to networks.
- **Bandwidth Management:** Controls data flow to prevent congestion.
- **Monitoring & Diagnostics:** Tracks traffic and helps troubleshoot network issues.



Architecture of Router

A typical router consists of:

- **Input Port**: Accepts packets, decapsulates them, and determines forwarding paths.
- **Switching Fabric:** The core of the router connecting input ports to output ports.

Can be implemented via:

- **Memory switching:** CPU copies packets to output ports.
- **Bus switching:** Single bus transfers packets to the correct port.
- **Interconnection networks:** Complex designs connecting multiple input/output ports.

Output Port: Transmits packets to outgoing links, managing queuing and link-layer functions.

Routing Processor: Executes routing protocols and algorithms, maintaining the forwarding table.

Common Routing Protocols

Open Shortest Path First (OSPF): Determines the optimal path across networks. Border Gateway Protocol **(BGP):** Shares routing information between edge routers.

Interior Gateway Routing Protocol (IGRP): Exchanges routing info within autonomous networks.

Enhanced IGRP (EIGRP): Requests routing paths from neighbors if unknown.

Exterior Gateway Protocol (EGP): Shares routing data between internet hosts.

Applications of Routers

- Connect remote servers, networks, and devices globally.
- Support wired and wireless communication, including high-speed data transfer.
- Used by ISPs to transmit audio, video, image, and email efficiently.
- Implement access control, enabling selective resource usage.

Types of Routers

- **Broadband Routers:** Connect computers to the Internet and share the connection.
- **Wireless Routers:** Create Wi-Fi networks in homes or offices.
- **Wired Routers:** Connect multiple devices via Ethernet cables, common in schools and offices.
- **Edge Routers:** Located at network boundaries, distributing packets to and from ISPs.
- **Core Routers:** Operate within networks, handling heavy data traffic.
- **Virtual Routers**: Software-based routers implemented on virtual machines for flexibility and scalability.
- **Portable Routers:** Small devices creating private Wi-Fi for mobility.

Security Challenges in Routers

- **Vulnerability Exploits:** Firmware flaws can be exploited by attackers; regular updates are necessary.
- **DDoS Attacks:** Distributed Denial-of-Service attacks can overload routers.
- **Default Admin Credentials:** Weak or unchanged credentials can allow unauthorized access.

How switches learn and use MAC address tables.

The MAC (Media Access Control) address is a 48-bit physical identity of a device used to uniquely identify a device in layer 2 of the OSI model. It is represented in hexadecimal. The 3 bytes of the address represent the OUI, and the last 3 bytes are unique to the device itself. The switch keeps track of the other Ethernet interfaces on the network to which it is linked in the MAC address table. Instead of broadcasting the data over all ports, the table allows the switch to send outgoing data (Ethernet frames) on the exact port needed to reach its destination (flooding). Two different types of entries are possible in the MAC address table:

- **Static:** The switch administrator must manually add static items to the table. Dynamic entries are less important than static entries. Static entries are active until the switch administrator deletes them.
- **Dynamic:** Through a process known as MAC learning, in which the switch retrieves the source MAC address (and VLAN ID, if present) of each Ethernet packet received on a port, dynamic entries are automatically added to the table. If the table does not already include the obtained address, it is added. Dynamic entries are automatically erased after being present in the table for a certain period of time (specified by the command mac-address-table age-time).

The switch stores the MAC information in a table called the CAM table or the MAC table.

CAM Table Stores:

- MAC address
- The interface
- VLAN MAC address belongs to

MAC address table for Cisco switches:

Consider a topology given below: Take 3 PC and connected them with a Cisco Switch and then connect the Switch to a router at the end such as given below:

Router0 configuration: Go to Router0 and do the following configuration.

```
Interface GigabitEthernet0/0/0

ip address 10.1.1.1 255.0.0.0

no shutdown
```